



Educación
Secretaría de Educación Pública



TECNOLÓGICO
NACIONAL DE MÉXICO®



TECNOLÓGICO NACIONAL DE MÉXICO INSTITUTO TECNOLÓGICO DE LEÓN

Tópicos para el despliegue de aplicaciones Proyecto final: RestAPI

QUE PRESENTA:

Cruz Valadez Montserrat
Herrera Villalpando Karen Alejandra
Prats Ramírez Nissi Sarahí



CARRERA:

INGENIERÍA EN SISTEMAS COMPUTACIONALES

CON LA ASESORÍA DE:

Levy Rojas Carlos Rafael

León, Guanajuato, Fecha: 02 de junio de 2026.



Índice

1. Análisis.....	7
1.1 Presentación del Proyecto	7
1.2 Factibilidad y beneficios	8
1.2.1 Introducción al estudio de factibilidad	8
1.2.2 Planteamiento del problema.....	8
1.2.3 Arquitectura general.....	9
1.2.4 Vista del flujo del sistema	13
1.2.5 Organigrama de actividades	15
1.2.6 Plan de tareas	16
1.2.7 Prototipo.....	17
1.2.8 Factibilidad.....	21
1.2.9 Conclusiones del análisis de factibilidad	22
1.3 Documento de especificación de requerimientos de software	23
1.3.1 Introducción	23
1.3.2 Descripción general	25
1.3.3 Requerimientos específicos	27
1.4 Priorización y plan de liberación	33
1.5 Descomposición del Sistema en Subsistemas.....	33
1.5.1 Subsistema de Presentación (Cliente de Escritorio).....	33
1.5.2 Subsistema de Lógica de Negocio (API REST en Go).....	34
1.5.3 Subsistema de Persistencia de Datos (Base de Datos).....	34
1.5.4 Subsistema de Orquestación (Infraestructura Docker)	34
1.6 Objetivos.....	34
1.7. Diccionario de Datos	35
1.8 Tablas y normalización	36
1.8.1. Tabla: employees	36
1.8.2. Tabla: usuarios.....	37

1.8.3. Tabla: titles	38
1.8.4. Tabla: salaries	38
1.9 DIAGRAMAS	39
1.9.1 Diagrama de clase.....	39
1.9.2 Diagrama Entidad Relación.....	40
1.10 Normalizaciòn	40
1.10.1 Primera Forma Normal (1FN)	41
1.10.2. Segunda Forma Normal (2FN)	41
1.10.3. Tercera Forma Normal (3FN).....	41
Pantalla de Inicio de Sesión.....	45
Pantalla Principal de Empleados	46
Pantalla de Registro de Empleados.....	47
Pantalla de Ediciòn de Empleados.....	48
Pantalla de Gestión de Usuarios.....	49
Interfaz en Modo Offline	50
Tecnologías Utilizadas	51
Aplicaciòn Android	51
Backend.....	52
Base de Datos	52
Arquitectura Implementada.....	52
Capa de Presentaciòn.....	52
Capa de Dominio	52
Capa de Datos.....	52
Implementaciòn del Backend	53
Implementaciòn del Sistema de Seguridad	53
Implementaciòn de Persistencia Local	53
Pruebas de Inicio de Sesión.....	54
Pruebas de Gestión de Empleados.....	54
Pruebas de Gestión de Usuarios.....	54
Pruebas de Control de Roles.....	54
Pruebas offline	55
Implantaciòn del Sistema.....	55

Resultados Obtenidos	55
4.3 Manual de Sistema	56
4.3.1 Introducción.....	56
4.3.2 Requisitos del Sistema	56
Requisitos de Hardware	56
Requisitos de Software	56
4.3.3 Instalación de la Aplicación	56
4.3.4 Inicio de Sesión.....	56
4.3.5 Pantalla Principal	57
4.3.6 Gestión de Empleados.....	57
Registrar Empleado.....	57
Consultar Empleados	57
Editar Empleado.....	57
Eliminar Empleado	58
4.3.7 Gestión de Usuarios.....	58
4.3.8 Roles del Sistema	58
Administrador	58
Empleado	58
4.3.9 Funcionamiento Sin Conexión	59
Funciones disponibles sin Internet	59
Funciones no disponibles sin Internet	59
4.3.10 Manejo de Errores	59
Error de conexión	59
Credenciales incorrectas	59
Error interno del servidor.....	59
4.3.11 Seguridad	60
4.3.12 Cierre de Sesión	60
4.3.13 Recomendaciones de Uso	60
4.3.14 Conclusión	60

Introducción

En el ámbito de la gestión empresarial, la administración eficiente del capital humano representa un pilar fundamental para la optimización operativa de cualquier negocio, incluyendo el sector de las cafeterías de especialidad. Históricamente, las micro y pequeñas empresas han dependido de metodologías tradicionales y manuales —tales como registros en papel, libretas u hojas de cálculo locales— para almacenar la información de su plantilla laboral. Sin embargo, estas prácticas suelen derivar en problemáticas críticas como la redundancia de datos, la pérdida de información por falta de respaldos centralizados y una notable lentitud burocrática al momento de procesar movimientos del personal, tales como altas, bajas o modificaciones salariales.

Con el objetivo de mitigar estas ineficiencias técnicas y operativas, el presente proyecto final expone el diseño, desarrollo y despliegue de **Coffeet**, un sistema de gestión de personal estructurado bajo una arquitectura cliente-servidor. La solución propuesta implementa de manera centralizada las cuatro operaciones fundamentales de persistencia de datos (CRUD: *Create, Read, Update, Delete*) para administrar de forma precisa e inmediata los atributos esenciales de los trabajadores, tales como sus nombres, puestos operativos (ej. baristas o cajeros) y salarios.

Desde la perspectiva arquitectónica, **Coffeet** se concibe mediante una separación formal de responsabilidades dividida en capas independientes:

- **Capa de Datos:** Gestionada por un sistema de bases de datos relacional MySQL, encargado de almacenar la información de manera persistente y normalizada hasta la Tercera Forma Normal (3^{er} Forma Normal).
- **Capa de Lógica de Negocio (Backend):** Consiste en un servidor robusto que expone una API REST construida en el lenguaje de programación Go (Golang), seleccionado por su alta eficiencia y óptimo manejo de peticiones HTTP en formato JSON.
- **Capa de Presentación (Frontend):** Constituida por una interfaz gráfica de usuario adaptada para interactuar de forma directa, ágil e intuitiva con el administrador del negocio.

Asimismo, alineado con los objetivos de la materia *Tópicos para el despliegue de aplicaciones*, la infraestructura del backend (servidor Go y base de datos MySQL) se encuentra completamente contenerizada y orquestada localmente mediante **Docker y Docker Compose**. Esto garantiza la portabilidad, el aislamiento técnico y la estandarización del entorno de ejecución sin depender de configuraciones globales en el sistema operativo anfitrión.

A través del desglose de este documento, se detallarán las etapas de análisis de requerimientos, el estudio de factibilidad técnica, económica y operativa, el diseño detallado de la base de datos, los diagramas de casos de uso y la estrategia de implementación offline. Con ello, se demuestra cómo la integración de tecnologías modernas de desarrollo y orquestación puede transformar un proceso administrativo obsoleto en una solución tecnológica automatizada, escalable y segura.

1. Análisis

1.1 Presentación del Proyecto

Nombre del Proyecto:

Coffeet: Sistema Cliente-Servidor para Administración de Empleados

Descripción (Planteamiento del Problema)

La administración de una cafetería de especialidad enfrenta desafíos significativos en la gestión de los datos de su personal. El método actual, manual y a menudo basado en documentos locales o de texto, genera ineficiencias, redundancia de datos y una falta de centralización en la información de los empleados. Esto resulta en dificultades técnicas para gestionar roles, actualizar salarios o procesar altas y bajas del personal de manera ágil desde diferentes equipos.

Este proyecto propone el desarrollo de **Coffeet**, un sistema estructurado bajo una arquitectura de **API REST**, que permita llevar un control preciso e inmediato de la plantilla laboral. El proyecto consiste en un servidor backend (desarrollado en Go) conectado a una base de datos, y un cliente de escritorio que consumirá los servicios sin requerir un sistema de autenticación complejo. El sistema incluirá funciones clave mediante operaciones CRUD:

- **Registro de empleados (Create):** El cliente de escritorio enviará los datos de un nuevo integrante (ej. baristas, cajeros) a la API para almacenar su nombre, puesto y salario en la base de datos.
- **Consulta de personal (Read):** El servidor responderá a las peticiones del cliente devolviendo la lista completa de empleados o los detalles de un trabajador específico en tiempo real.
- **Actualización de datos (Update):** A través de la API, se facilitará la modificación de la información del empleado en caso de ascensos, cambios de puesto o ajustes salariales.
- **Baja de personal (Delete):** El sistema gestionará la eliminación de los registros en la base de datos cuando un empleado termine su relación laboral mediante una petición desde la interfaz gráfica.

El objetivo es aplicar los principios de despliegue de aplicaciones construyendo un servidor robusto y una aplicación cliente que se comuniquen eficientemente, optimizando la administración técnica de los datos del personal.

A Quién Va Dirigido

Este proyecto está diseñado para la administración de cafeterías que manejan una plantilla de empleados con múltiples roles y necesitan una solución tecnológica centralizada. Es ideal para demostrar cómo modernizar la gestión de personal mediante el uso de una arquitectura basada en **servicios REST**. El sistema está pensado para ejemplificar la comunicación técnica y directa entre una aplicación de escritorio (interfaz de usuario) y un servidor backend,

facilitando la manipulación de la base de datos de forma eficiente y escalable, eliminando la dependencia de métodos de registro manuales aislados.

1.2 Factibilidad y beneficios

1.2.1 Introducción al estudio de factibilidad

La factibilidad en proyectos de software hace referencia al grado en que un sistema puede desarrollarse e implementarse con éxito, considerando aspectos técnicos, económicos y operativos, así como los recursos disponibles en la organización. El estudio de factibilidad busca responder una pregunta importante: ¿es viable llevar a cabo el proyecto en las condiciones actuales y con los recursos existentes?

La factibilidad en proyectos de software hace referencia al grado en que un sistema puede desarrollarse e implementarse con éxito, considerando aspectos técnicos, económicos y operativos, así como los recursos disponibles en la organización. El estudio de factibilidad busca responder una pregunta importante: ¿es viable llevar a cabo el proyecto en las condiciones actuales y con los recursos existentes?

En el caso de Coffeet, este estudio permitirá determinar si es factible desarrollar un sistema basado en arquitectura cliente-servidor diseñado específicamente para apoyar a la administración de una cafetería de especialidad en el control y gestión de los datos de sus empleados (altas, bajas, actualizaciones de salarios y puestos).

El sistema busca sustituir el actual método manual basado en registros físicos o documentos de texto dispersos, el cual presenta múltiples problemas como redundancia de datos, pérdida de información y falta de centralización.

El análisis de factibilidad es importante porque permitirá evaluar:

- **La viabilidad técnica**, considerando la disponibilidad de software (Go, Docker, MySQL), hardware y tecnologías necesarias para implementar la API REST y el cliente de escritorio.
- **La viabilidad económica**, tomando en cuenta los costos de desarrollo y operación frente a los beneficios tangibles e intangibles que generará.
- **La viabilidad operativa**, garantizando que el usuario principal (el administrador) podrá utilizar la interfaz de escritorio para gestionar la base de datos sin necesidad de conocimientos técnicos avanzados.

Objetivo del estudio: analizar la viabilidad técnica, económica y operativa del sistema Coffeet, asegurando que la propuesta no solo sea posible en términos de despliegue y arquitectura, sino también útil y ágil en la práctica cotidiana de la cafetería.

1.2.2 Planteamiento del problema

Actualmente, la administración lleva el control del personal de manera manual, utilizando libretas, hojas de cálculo sin sincronizar y apuntes dispersos. Este método genera problemas que afectan directamente la gestión de recursos humanos del negocio:

- **Errores de registro**, como información duplicada sobre los turnos o salarios desactualizados.
- **Pérdida de información**, debido a que los documentos físicos se dañan o los archivos locales no tienen un respaldo central.
- **Gestión burocrática lenta**, lo que impide actualizar con agilidad los ascensos, cambios de rol (ej. de auxiliar a barista) o las bajas definitivas.
- Esta situación repercute en una mala organización interna y descontrol en la plantilla laboral. Por lo tanto, se identifica la necesidad de implementar un sistema de gestión de personal (RestApp) que permita:
 - Registrar y mantener un directorio actualizado de empleados (Create).
 - Consultar en tiempo real la lista completa del personal activo (Read).
 - Actualizar sueldos, puestos o datos específicos de manera inmediata (Update).
 - Dar de baja del sistema a los empleados que terminan su contrato (Delete).

El sistema Coffeet surge como una solución tecnológica diseñada para automatizar esta gestión, proporcionando una interfaz de escritorio amigable que se comunica directamente con un servidor central.

1.2.3 Arquitectura general

El sistema Coffeet se propone como una aplicación de gestión de personal con una arquitectura cliente-servidor. La idea central es que el administrador pueda acceder desde una aplicación de escritorio local, mientras que la lógica y los datos se gestionan en un servidor (API REST) operando en contenedores.

Componentes principales:

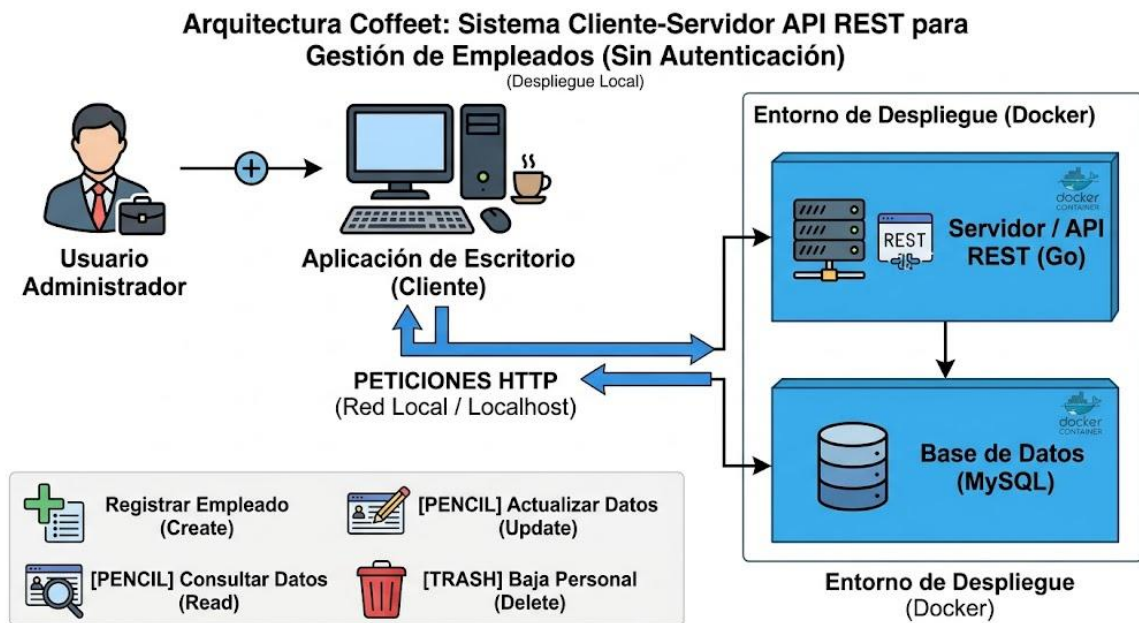
- **Despliegue (Orquestación):** Docker, para contenerizar tanto la base de datos como el servidor, garantizando que el sistema pueda levantarse en cualquier entorno operativo.
- **Base de Datos:** MySQL, que permite una gestión confiable, relacional y estructurada de los datos de los empleados.
- **Backend (Servidor/API REST):** Desarrollado en Golang (Go), elegido por su alta eficiencia, compilación rápida y excelente manejo de peticiones HTTP para exponer los endpoints del CRUD.
- **Frontend (Cliente):** Aplicación de escritorio desarrollada con interfaz gráfica, encargada de consumir los datos de la API mediante peticiones de red.
- **Restricciones de diseño:** Sistema simple, enfocado a la manipulación de datos sin módulo de autenticación complejo.

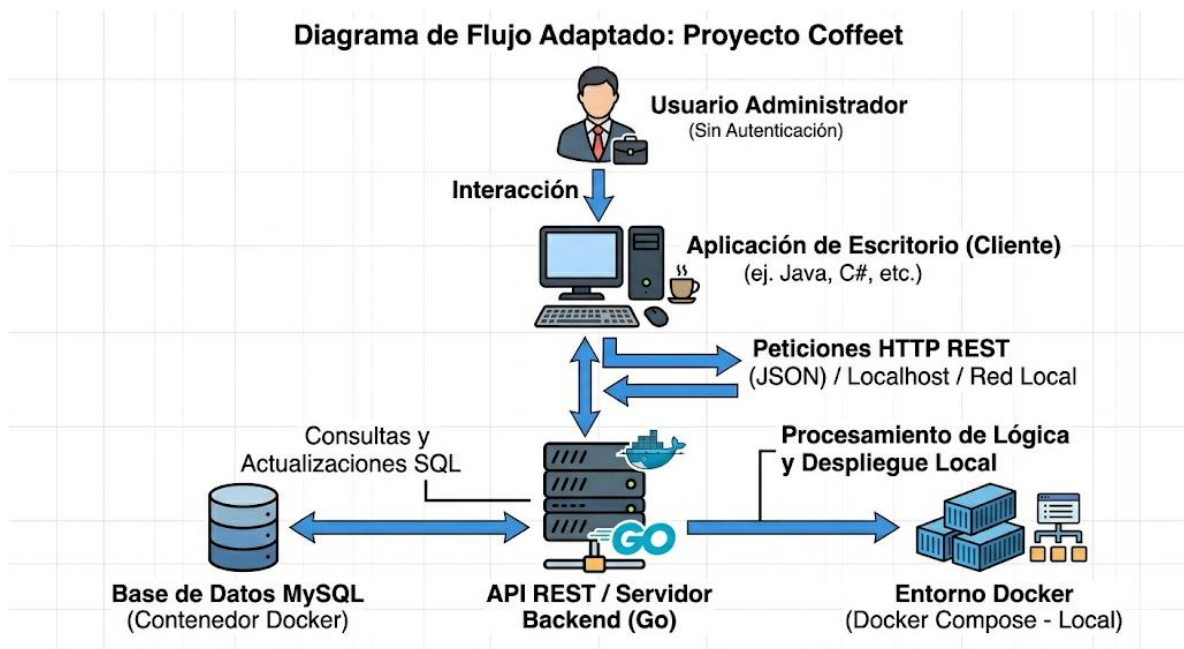
Esquema general de funcionamiento:

- El administrador ejecuta la aplicación de escritorio (cliente) teniendo acceso directo al sistema sin requerir autenticación.
- Registra a los nuevos empleados en el sistema (**Create**), ingresando datos como nombre completo, puesto (ej. barista, cajero) y salario.
- Consulta la lista completa del personal activo en tiempo real (**Read**), visualizando la información extraída directamente del servidor.

- Actualiza los perfiles de los empleados existentes (**Update**), modificando salarios o cambiando roles según sea necesario.
- Registra las bajas definitivas del personal cuando terminan su relación laboral con la cafetería (**Delete**).

En cada interacción, la aplicación de escritorio se comunica con la API REST (servidor) mediante peticiones HTTP para mantener la base de datos centralizada y sincronizada de forma automática.





1.2.3.1 Consideraciones de software

Para seleccionar las tecnologías adecuadas al proyecto de despliegue, se analizaron distintas alternativas para el servidor y el cliente.

1.3.1.1 Comparación y elección del software

- **Lenguaje de Programación del Servidor (Go vs Rust)** El maestro solicitó explícitamente el uso de uno de estos dos lenguajes modernos.
 - **Rust:** Ofrece un rendimiento insuperable y seguridad de memoria garantizada, pero posee una curva de aprendizaje sumamente pronunciada y una sintaxis compleja que excede las necesidades de un CRUD simple.
 - **Golang (Go):** Diseñado por Google para la creación de APIs y servidores concurrentes. Su sintaxis es limpia, la curva de aprendizaje es amigable y genera binarios ligeros ideales para ser contenerizados en Docker.

Decisión: Se selecciona **Go** como lenguaje backend por su simplicidad operativa y facilidad para levantar servicios RESTful rápidos.

- **Manejador de la Base de Datos (SQL Server vs MySQL)**
 - **SQL Server:** Robusto, pero requiere licencias costosas o versiones express limitadas, además de ser más pesado para contenerizar rápidamente.

- **MySQL:** Código abierto, eficiente, altamente compatible con Go mediante drivers estándar y con imágenes oficiales ligeras en Docker.

Decisión: Se selecciona **MySQL** por su escalabilidad, costo nulo y facilidad de integración.

- **Desarrollo de la Interfaz (Cliente de Escritorio)** La interfaz será la herramienta directa del administrador.

Decisión: Se utilizará un lenguaje de propósito general con soporte para interfaces gráficas (ej. Java con Swing/JavaFX) capaz de realizar peticiones HTTP (GET, POST, PUT, DELETE) hacia el servidor en Go.

En conclusión, la elección tecnológica para Coffeet será:

- **Despliegue:** Docker (Docker Compose).
- **Base de datos:** MySQL.
- **Lenguaje backend:** Golang (Go).
- **Frontend:** Aplicación de Escritorio.



1.2.3.2 Consideraciones de hardware

El sistema Coffeet se implementará probando el cliente y el servidor en el mismo entorno local o red de área local (LAN). Para garantizar el correcto desempeño al ejecutar los contenedores de Docker y la aplicación cliente simultáneamente, se consideran las siguientes especificaciones recomendadas para el equipo de cómputo:

- **Procesador (CPU):** Intel i5 / AMD Ryzen 5 o superior, con soporte para virtualización (necesario para Docker).

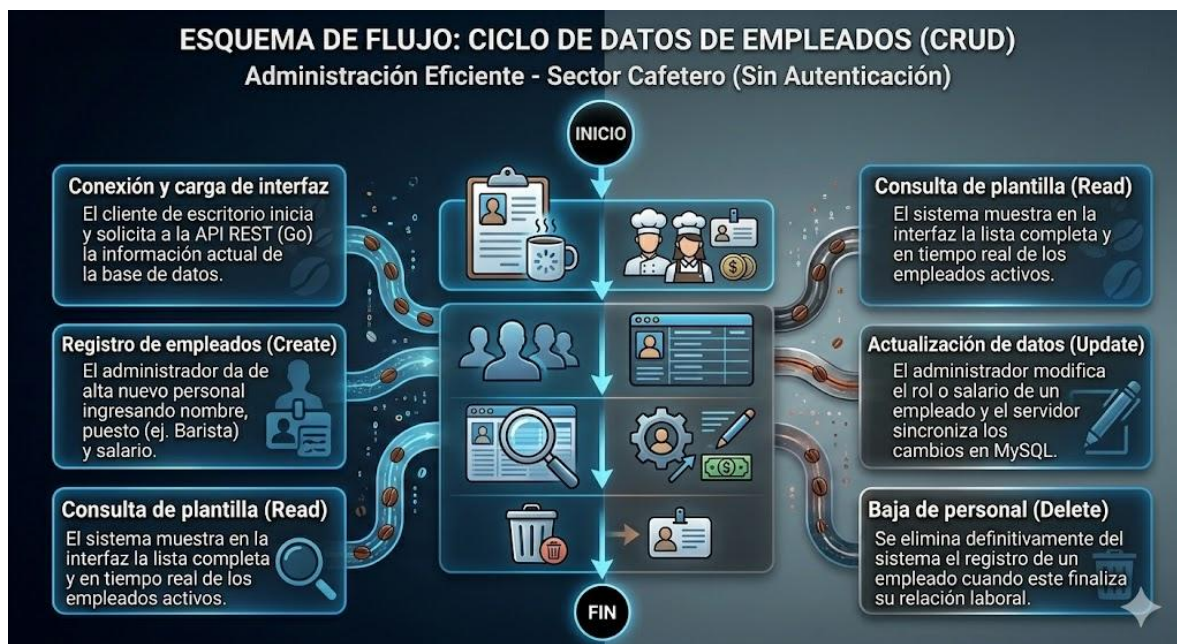
- **Memoria RAM:** Mínimo 8 GB, recomendable 16 GB, para soportar el motor de Docker y la máquina virtual del cliente de escritorio sin afectar el rendimiento.
- **Almacenamiento:** SSD de 256 GB o superior para agilizar la lectura de las imágenes de los contenedores.
- **Conectividad:** Tarjeta de red estándar para la comunicación en *localhost*.

1.2.4 Vista del flujo del sistema

El flujo de información del sistema, basado en las operaciones solicitadas (CRUD), se estructura en los siguientes pasos:

1. **Apertura del Cliente:** El administrador ejecuta la aplicación de escritorio.
2. **Registro de Empleados (Create):** Se ingresan los datos del nuevo integrante (nombre, puesto, salario) en un formulario y se envían a la API REST.
3. **Consulta de Plantilla (Read):** El cliente solicita a la API la lista de empleados y la muestra en una tabla dinámica en pantalla.
4. **Actualización de Datos (Update):** El administrador selecciona a un empleado, modifica su salario o puesto y guarda los cambios, enviando la actualización al servidor.
5. **Baja de Personal (Delete):** Se elimina el registro de un empleado de la base de datos mediante una petición a la API.

Tabla de actividades:





1.2.5 Organigrama de actividades

El desarrollo del sistema Coffeet se llevará a cabo en cinco etapas:

1. **Diseño de la base de datos:** Creación del diagrama Entidad-Relación y script SQL para la tabla de empleados.
2. **Desarrollo de la API (Backend):** Programación en Go de las rutas y la conexión a MySQL.
3. **Desarrollo del Cliente (Frontend):** Diseño y programación de las pantallas de la aplicación de escritorio.
4. **Integración y Pruebas:** Conexión del cliente con la API y verificación del CRUD completo.
5. **Despliegue:** Creación de los archivos Dockerfile y docker-compose.yml para levantar los servicios.

1.2.5.1 Organización de las actividades

ETAPA	ACTIVIDAD	TIEMPO ASIGNADO (HRS)
Análisis de requerimientos	Recopilación de información sobre el proceso actual de administración de personal en la cafetería.	8
	Definición de los casos de uso (CRUD) para la gestión de empleados (ej. Registrar barista, actualizar salario).	6
	Análisis de infraestructura local y requerimientos técnicos para el uso de contenedores (Docker).	10
	Definición de requerimientos funcionales (operaciones de la API) y no funcionales (tiempos de respuesta).	16
Diseño del sistema	Diseño de la arquitectura Cliente-Servidor (API REST en backend y Aplicación de Escritorio en frontend).	12
	Diseño de la base de datos (diagrama Entidad-Relación y definición de la tabla de empleados).	12
	Diseño de interfaces gráficas para el cliente local (pantalla principal, formularios de registro y edición).	20

	Especificación de las rutas y endpoints de la API (métodos GET, POST, PUT, DELETE).	16
Desarrollo	Construcción de la base de datos MySQL y estructuración de los campos requeridos.	16
	Desarrollo del servidor backend (API REST) utilizando el lenguaje Go (Golang).	32
	Programación de la interfaz gráfica del cliente de escritorio (ej. utilizando Java).	32
	Integración: Conexión de la aplicación de escritorio con el servidor para el consumo de la API REST.	32
Pruebas y corrección de errores	Pruebas unitarias de los endpoints del servidor (simulando peticiones independientes a la API).	16
	Pruebas funcionales del CRUD completo desde el cliente de escritorio hacia la base de datos.	20
	Pruebas de orquestación y comunicación en red local (Localhost) entre los diferentes componentes.	16
	Corrección de errores (debugging) detectados en la lógica de Go, las peticiones HTTP o la interfaz.	40
Instalación y despliegue	Creación de archivos Dockerfile y configuración del entorno con docker-compose.yml.	10
	Levantamiento de los servicios (Contenedor de Base de Datos y Contenedor del Servidor Go) vía Docker.	10
	Ejecución de la aplicación de escritorio en el equipo del administrador y pruebas finales de conexión.	8

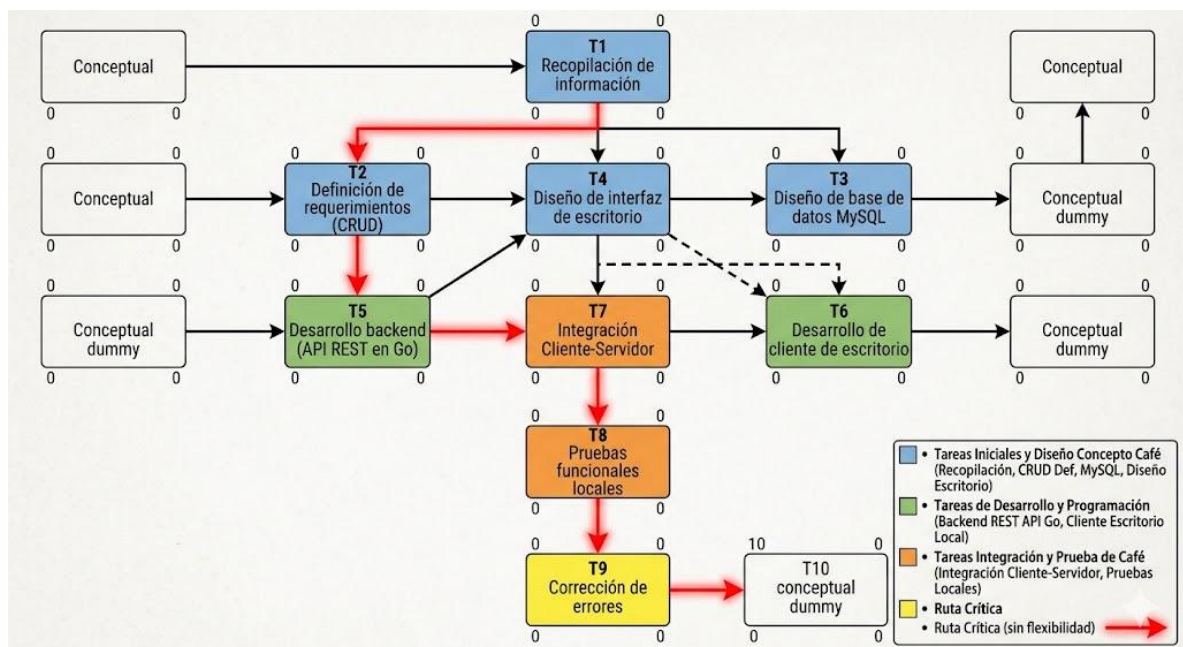
1.2.6 Plan de tareas

1.2.6.1 Duración de tareas y dependencia

T	TAREA	DURACIÓN ESTIMADA	DEPENDENCIA
T1	Recopilación de información	2 días	Ninguna
T2	Definición de requerimientos (CRUD)	3 días	Recopilación de información
T3	Diseño de base de datos MySQL	4 días	Definición de requerimientos
T4	Diseño de interfaz de escritorio	4 días	Requerimientos definidos

T5	Desarrollo backend (API REST en Go)	8 días	Diseño de BD e interfaces
T6	Desarrollo de cliente de escritorio	8 días	Diseño de interfaces
T7	Integración Cliente-Servidor	5 días	Desarrollo backend/cliente
T8	Pruebas funcionales locales	5 días	Integración completada
T9	Corrección de errores	5 días	Pruebas realizadas
T10	Instalación (Docker) y capacitación	4 días	Sistema validado

1.2.6.2 Red de actividades



1.2.7 Prototipo

El prototipo del sistema Coffeet consistirá en un modelo visual de la interfaz de escritorio.

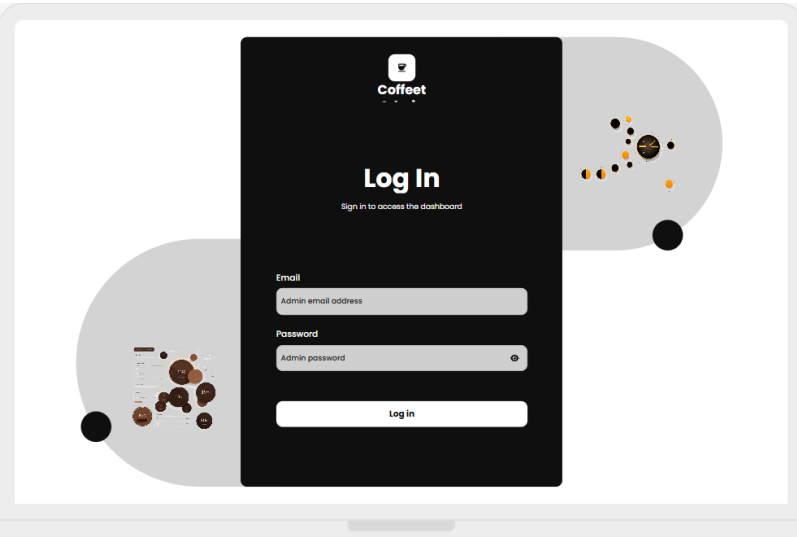
1.2.7.1 Interfaz de usuario

El prototipo incluirá las siguientes vistas:

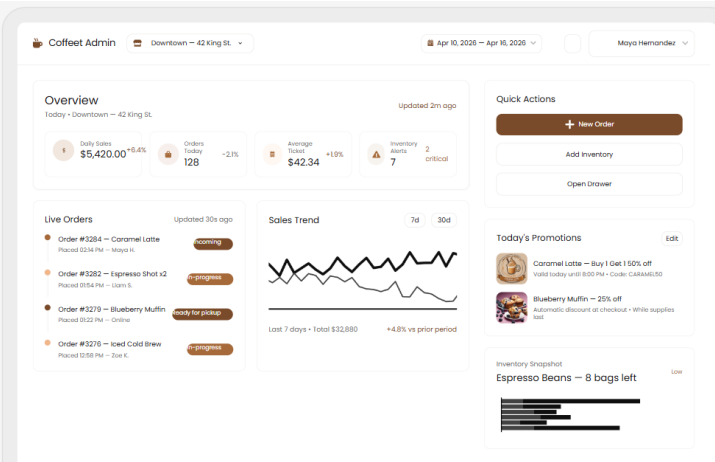
- **Pantalla Principal (Dashboard):** Vista general con la lista de todos los empleados activos (tabla de datos).
- **Panel de Registro:** Un formulario simple con campos de texto para "Nombre", "Puesto" y "Salario" con un botón de "Guardar".
- **Controles de Edición:** Botones de "Editar" y "Eliminar" vinculados a cada registro de la tabla.

Las siguientes imágenes muestran una idea de la interfaz que se mostrara al usuario final del sistema.

Interactive prototype



Additional design proposals



Orders & POS - Coffeet Admin

Coffeet Admin

Orders & POS

Terminal 3 • Floor 1

Global search orders

Ava Martinez
Manager

Categories

Quick access

Hot Beverages

Cold Drinks

Pastries

Sandwiches

Extras

Search products (e.g.,)

Clear

Dine-in

Takeaway

Delivery

Sort: Most Popular



Cappuccino

\$3.50

Add



Iced Latte

\$4.00

Add



Almond Croissant

\$2.75

Add



Turkey Club

\$7.25

Add



Vanilla Cold Brew

\$4.50

Add



Extra Shot

\$0.75

Add

Views

Grid3 cols

List

Order #

#0427 • Dine-in

2 items • Seat 4

Cappuccino

Oat milk • Extra foam

Modifiers: + Extra shot

- 1 +

Void

Notes

\$3.50

Almond Croissant

Toasted • Extra almond

- 1 +

Void

Notes

\$2.75

Subtotal

\$6.25

Tax

\$0.50

Service

\$0.00

Total

\$6.75

Hold

Pay

Print Receipt

Mark Ready

Tender

Cash

Card

Account

Discount

Apply

e.g., 10% or 1.50

Confirm

Tip: Use keyboard numbers to update quantities. Press Enter to focus Pay. Use Ctrl+P to print.

Inventory - Coffeet Admin

Coffeet Admin

DashboardOrders & POSInventoryStaff & SchedulingReports & Settings

Admin • coffeet@cofast.co

Add ItemImport CSVAll Categories

Search items, SKU, supplier...

Bulk EditShowing 1-5 of 5

2 items are below their reorder thresholds. Suggested reorder quantities are available in item details.

View Low Stock

Create Purchase Order

Item	SKU	Current Qty	Unit	Reorder Threshold	Supplier	Actions
 Colombian Medium Roast Beans Whole bean, medium profile, balanced acidity	CB-0012	12	kg	10	RoastHouse Supplies	Order Details
 Vanilla Bean Syrup 500ml concentration, classic vanilla	SV-045	4	bottle	8	SweetCraft Ingredients	Order Details
 Oat Milk - Barista Blend 1L cartons, creamy texture for frothing	OM-220	28	carton	12	DairyFree Co.	Order Details
 Compostable Cups - 12oz Pack of 100 cups with lids available separately	PC-12	18	pack	20	EcoServe Ltd.	Order Details
 Dutch Processed Cocoa Powder Dark, rich cocoa for mocha beverages	CK-330	52	kg	8	ChocoSuppliers Inc.	Order Details

5 items shown • 2 low-stock

Prev

Next

Vanilla Bean Syrup

SKU SV-045 • Supplier: SweetCraft Ingredients

30-day usage



Recent Purchase Orders

 PO #2342
Received • 2025-04-12 • 12 bottles

Delivered

 PO #2278
Placed • 2025-03-02 • 24 bottles

Placed

Suggested Reorder

Based on average daily usage and lead time

Lead time: 7 days

20

Quick Reorder

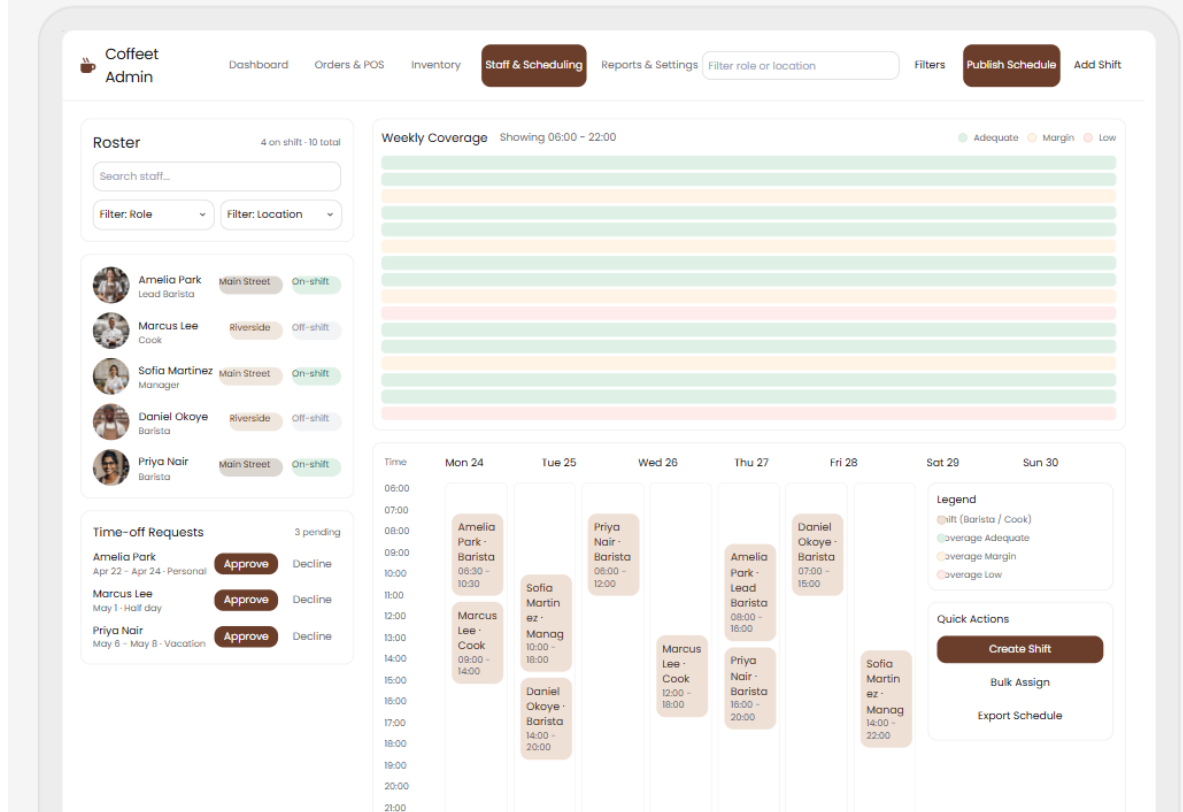
Customize

Last restock: 2025-04-12

Average daily usage: 0.6 bottles/day

Suggested reorder frequency: every 30 days

Staff & Scheduling - Coffeet Admin



1.2.7.2 Publico general

De acuerdo a las restricciones del proyecto, el sistema será simple y **no contará con autenticación**. Al iniciar la aplicación de escritorio, el usuario tendrá acceso directo a la manipulación de los datos.

1.2.8 Factibilidad

1.2.8.1 Factibilidad técnica

Desde el punto de vista técnico, Coffeet es completamente factible. Las herramientas (Go, Docker, MySQL) son de uso libre, cuentan con amplia documentación y el equipo de cómputo actual soporta los requerimientos de virtualización. La separación entre cliente y servidor garantiza una arquitectura limpia.

1.2.8.2 Factibilidad económica

Al ser un proyecto desarrollado académicamente, los costos de mano de obra y licenciamiento de software son nulos. Las herramientas seleccionadas son de código abierto o gratuitas. Los únicos gastos asociados son los costos operativos del ambiente de desarrollo (energía eléctrica, internet).

1.2.8.3 Factibilidad operativa

El sistema será operado mediante una interfaz de escritorio diseñada para ser minimalista. Para garantizar la viabilidad operativa se han considerado:

- **Interfaz directa:** Al no haber inicio de sesión, las operaciones son inmediatas.
- **Comunicación transparente:** El usuario solo interactúa con botones y formularios; la complejidad de la red, Go y Docker quedan invisibles en el servidor de fondo.

1.2.8.4 Riesgos

En todo proyecto tecnológico existen riesgos que deben considerarse:

- **Fallas en la ejecución de contenedores:** Si el servicio de Docker se detiene, el cliente de escritorio no podrá conectar con la base de datos.
- **Seguridad de los datos:** Debido a la falta intencionada de un sistema de autenticación, cualquier persona con acceso a la computadora donde esté el cliente podría modificar los registros.

1.2.8.5 Ventajas y desventajas

Ventajas:

- Arquitectura moderna y escalable (separación total entre Frontend y Backend).
- Portabilidad del servidor y base de datos gracias a Docker.
- Velocidad de procesamiento de datos por el uso de Go.

Desventajas:

- Vulnerabilidad de acceso al no contar con perfiles de usuario y contraseñas.

1.2.8.5.1 Beneficios tangibles e intangibles

- **Tangibles:** Control exacto y digitalizado del número de empleados, sus roles y salarios. Eliminación total de formatos en papel.
- **Intangibles:** Experiencia práctica en el desarrollo y orquestación de microservicios o arquitecturas distribuidas, y modernización de la gestión interna.

1.2.9 Conclusiones del análisis de factibilidad

Tras realizar el análisis, se concluye que el proyecto **Coffeet** es viable y beneficioso en todos sus aspectos:

- **Técnico:** Se cumple con los lenguajes y arquitecturas requeridos (API REST, Go, App de Escritorio, base de datos).
- **Económico:** La inversión se limita al tiempo de codificación y pruebas, sin costo de licencias.
- **Operativo:** La gestión a través de una aplicación local simplifica en gran medida las tareas administrativas.

Por todo lo anterior, se determina que el proyecto es factible y listo para su fase de programación y despliegue.

1.3 Documento de especificación de requerimientos de software

1.3.1 Introducción

1.3.1.1 Propósito

El propósito de este documento es proporcionar una descripción detallada del sistema cliente-servidor “Coffeet”. Se muestra el propósito y una declaración completa que permita el desarrollo del sistema de gestión de empleados enfocado en el sector cafetero. La intención principal es generar una especificación clara que sirva como referencia para el desarrollo, asegurando que el producto final cumpla con las necesidades específicas de la administración del negocio. Este documento explica el alcance, las limitaciones, las interfaces y las interacciones del sistema.

1.3.1.2 Alcance

“Coffeet” es un sistema cliente-servidor de gestión de personal desarrollado para optimizar la administración en una cafetería. Su función principal es digitalizar y centralizar el control de la plantilla laboral, permitiendo el registro, consulta, edición y baja precisa de los trabajadores (operaciones CRUD) a través de una aplicación de escritorio local de acceso directo, omitiendo módulos de autenticación complejos. El sistema mejorará la precisión de los registros de nómina o puestos (como baristas o cajeros) y agilizará los movimientos internos del personal.

El sistema está diseñado para un despliegue y funcionamiento puramente local (Localhost o red interna). Toda la información se almacena en una base de datos MySQL alojada en un contenedor independiente, mientras que la lógica de negocio se procesa a través de un servidor backend con una API REST desarrollada en Go, también contenerizada. El administrador interactuará con el sistema exclusivamente mediante la interfaz gráfica de la aplicación de escritorio, la cual consume los servicios de la API mediante peticiones HTTP en formato JSON.

Desglosándose en los siguientes puntos:

- Digitalizar y centralizar el control de la plantilla laboral (empleados de la cafetería).
- Permitir el alta y registro inicial de nuevos empleados (Create), capturando datos clave como nombre completo, puesto y salario.
- Proporcionar una consulta dinámica y visualización en tiempo real (Read) de la lista completa de personal activo en el sistema.

- Permitir la modificación y actualización integral de los perfiles (Update) para ajustar salarios, puestos o departamentos.
- Registrar la baja definitiva de los registros de base de datos (Delete) cuando el personal concluye su relación laboral.
- Permitir el acceso directo a las funciones administrativas del sistema sin requerir un inicio de sesión o credenciales de usuario.
- Comunicar los componentes mediante un flujo local de peticiones HTTP utilizando JSON para el intercambio de datos entre el cliente de escritorio y el servidor.
- Orquestar y desplegar toda la arquitectura del backend (Servidor Go y Base de Datos MySQL) dentro de un entorno local contenerizado mediante herramientas de Docker.

1.3.1.3 Definiciones, acrónimos y abreviaturas

Término	Definición
Administrador	Usuario con acceso a todas las funcionalidades del sistema (ej. Gerente de la cafetería), sin requerir proceso de autenticación.
API REST	Interfaz de Programación de Aplicaciones que permite la comunicación entre la aplicación de escritorio y el servidor para la transferencia de datos.
CRUD	Acrónimo en inglés de Create, Read, Update, Delete (Crear, Leer, Actualizar, Eliminar). Representa las operaciones básicas de gestión de registros en la base de datos.
Alta de personal	Registro de ingreso de un nuevo empleado (ej. Barista, Cajero) a la base de datos de la cafetería.
Baja de personal	Eliminación definitiva del registro de un empleado en el sistema cuando finaliza su relación laboral.
Dashboard (Panel Principal)	Pantalla principal de la aplicación de escritorio que muestra la lista de la plantilla laboral activa y botones de acción.
Docker / Contenedor	Tecnología que permite empaquetar, aislar y ejecutar el servidor (Go) y la base de datos (MySQL) en un entorno local estandarizado.
JSON	(JavaScript Object Notation) Formato de texto ligero utilizado para el intercambio de datos entre la aplicación cliente y la API REST.
Localhost	Entorno de red local del equipo donde se despliega y ejecuta el sistema, sin necesidad de conexión a un servidor web externo.
Stakeholders	Cualquier persona u organización que interactúa con el sistema o tiene interés directo en el proyecto (ej. Dueño de Coffeet).

1.3.2 Descripción general

Coffeet es un sistema cliente-servidor local e independiente. Su función central es interactuar con una base de datos que almacena toda la información relacionada con la plantilla laboral. El usuario administrador interactúa con el sistema a través de una aplicación de escritorio, la cual se comunica localmente con un servidor backend que aloja la lógica de negocio (API REST) y gestiona el acceso a la base de datos centralizada.

1.3.2.1 Perspectiva del producto

Coffeet es un sistema que opera bajo una arquitectura cliente-servidor local estructurada en tres capas bien definidas. La capa de presentación consiste en una aplicación nativa de escritorio desarrollada con interfaz gráfica (ej. Java), que se ejecuta directamente en el equipo del administrador y permite un acceso directo a las funciones sin necesidad de un proceso de inicio de sesión. La capa de lógica de negocio está implementada como una API REST utilizando el lenguaje Go (Golang) y reside localmente dentro de un entorno contenerizado. Esta capa recibe y procesa todas las solicitudes del cliente mediante peticiones HTTP, aplica las reglas de negocio correspondientes a la gestión de personal (operaciones CRUD) y devuelve las respuestas en formato JSON. Finalmente, la capa de datos está gestionada por un sistema de gestión de bases de datos MySQL, alojado en un contenedor independiente, encargado de almacenar de manera persistente la información de los empleados (nombres, puestos y salarios).

El sistema se ejecuta completamente en un entorno local (Localhost), por lo que no requiere de una conexión a Internet para su funcionamiento operativo. Su integración técnica depende directamente del hardware del equipo del usuario final para el correcto despliegue de la interfaz de escritorio y para soportar la ejecución del motor de Docker (Docker Compose), el cual se encarga de orquestar de forma centralizada tanto el servidor backend en Go como la base de datos MySQL.

1.3.2.2 Funciones del product

El producto debe cumplir con las siguientes funciones organizadas por módulos:

- **Módulo de Gestión de Empleados (CRUD):** Es el núcleo del sistema. Brinda una interfaz centralizada en la aplicación de escritorio para el mantenimiento de la información de la plantilla laboral. Esto incluye la creación (alta de nuevos baristas/cajeros), consulta (visualización de la lista activa), actualización (modificación de salarios o puestos) y eliminación (bajas definitivas) de los registros.
- **Módulo de Comunicación y Sincronización:** Gestiona las peticiones HTTP en formato JSON desde el cliente de escritorio hacia la API REST en Go, garantizando que cada operación realizada en la interfaz se refleje instantáneamente en la base de datos MySQL.
- **Módulo de Despliegue Local:** Asegura la inicialización y orquestación del entorno de backend mediante Docker, levantando de manera conjunta y automatizada los contenedores del servidor y la base de datos.

1.3.2.3 Limitaciones

- **Limitación de Entorno (Conectividad):** El sistema está diseñado estrictamente para un entorno local (Localhost). No requiere conexión a Internet, pero por lo mismo, no es accesible de forma remota desde ubicaciones externas a la red de la cafetería.
- **Limitación de Seguridad:** Debido a la naturaleza de los requerimientos, el sistema carece de un módulo de autenticación (Login). La seguridad y privacidad de los datos dependen de las medidas de seguridad físicas y de control de acceso del sistema operativo de la computadora donde se instala.
- **Limitación de Dispositivo:** El sistema consta de una aplicación nativa de escritorio. No cuenta con versión web ni es compatible con dispositivos móviles (smartphones o tablets).

Desglosado en los siguientes puntos:

- **No es un sistema de control de asistencia:** No gestiona horarios, retardos, horas extra ni checadores biométricos.
- **No calcula nóminas fiscales:** Registra el salario base, pero no realiza cálculos de retenciones de impuestos, seguridad social ni genera recibos de nómina timbrados.
- **No es un punto de venta (POS) ni inventario:** Este sistema se limita exclusivamente a la gestión de empleados; no controla ventas de café, stock de insumos ni manejo de caja.

1.3.2.5 Suposiciones y dependencias

Suposición de Infraestructura: Se supone que la computadora anfitriona en la cafetería cumple con los requisitos mínimos de hardware y tiene instalado el motor de contenedores (Docker Desktop/Docker Engine) necesario para ejecutar el servidor Go y MySQL.

Suposición de Conocimiento del Usuario: Se asume que el usuario final (Carlos) tiene competencias digitales básicas para operar aplicaciones de escritorio en su sistema operativo, minimizando la curva de aprendizaje.

Dependencia de Puertos Locales: La correcta comunicación entre la aplicación de escritorio y la API REST depende de que los puertos designados en Localhost (ej. puerto 8080 para Go, 3306 para MySQL) no estén bloqueados por firewalls ni ocupados por otros programas.

Dependencia de Sistema Operativo: La aplicación cliente requiere ser ejecutada en un sistema operativo compatible con el lenguaje en el que fue desarrollada (ej. Windows si se desarrolla en C# o con máquina virtual Java si es el caso).

1.3.2.6 Atención especial a requerimientos

Se reconoce que el desarrollo de software es un proceso iterativo y sujeto a imprevistos técnicos, especialmente al integrar múltiples tecnologías (Cliente de escritorio, Go, Docker, MySQL). Por lo tanto, se ha establecido una clara priorización de requisitos.

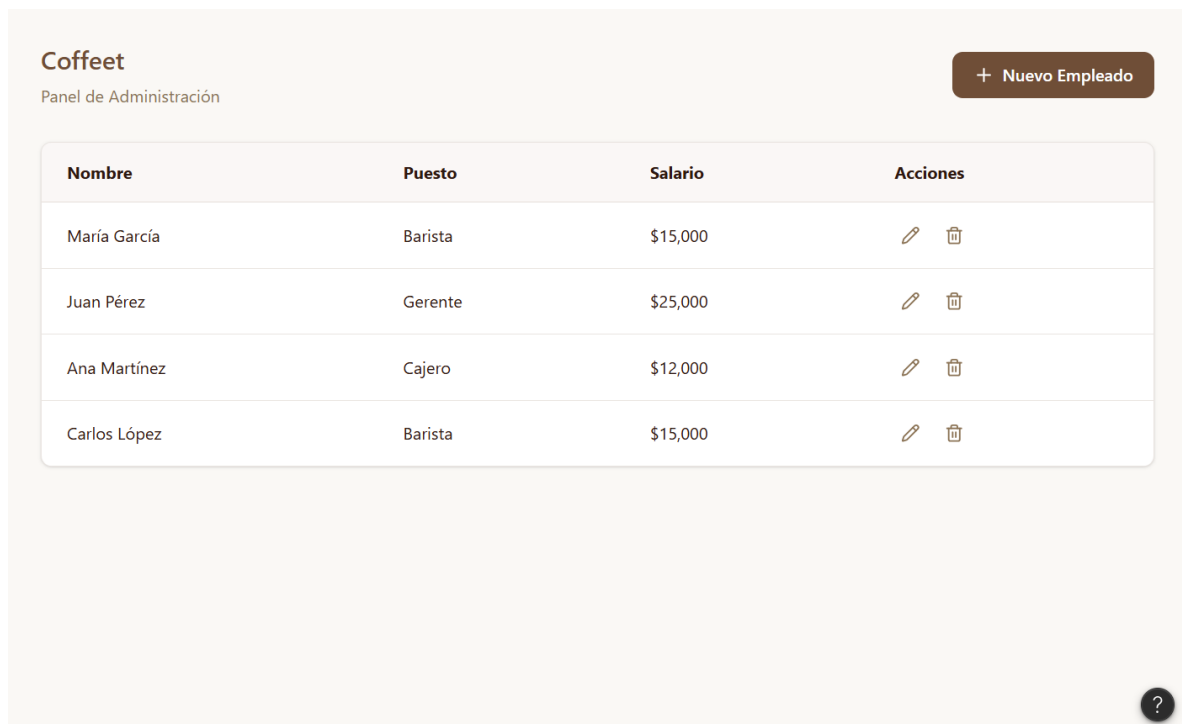
En caso de que surjan impedimentos que pongan en riesgo la línea de tiempo del proyecto, se adoptará la siguiente estrategia: Los requisitos de prioridad ALTA (funcionalidad completa de las 4 operaciones CRUD y el levantamiento de los contenedores) son innegociables para la entrega final. Los requisitos de prioridad MEDIA (validaciones complejas de campos en la interfaz o alertas de interfaz) serán el foco secundario. Los requisitos de prioridad BAJA (búsquedas por filtros avanzados o exportación de tablas) serán los primeros en ser considerados para su postergación. Esta estrategia garantiza que el sistema se entregue con la arquitectura técnica exigida y la funcionalidad esencial operando correctamente.

1.3.3 Requerimientos específicos

Esta sección contiene los requerimientos funcionales y de calidad del sistema, aportando descripciones de las interfaces de software, de hardware y de comunicaciones, y proporciona un prototipo básico de la interfaz de usuario.

1.3.3.1 Interfaz de Usuario

.....



The screenshot displays the 'Coffeet' administration interface. At the top left, the title 'Coffeet' is followed by 'Panel de Administración'. On the top right, there is a button labeled '+ Nuevo Empleado'. The main content is a table with four columns: 'Nombre', 'Puesto', 'Salario', and 'Acciones'. The table lists four employees: María García (Barista, \$15,000), Juan Pérez (Gerente, \$25,000), Ana Martínez (Cajero, \$12,000), and Carlos López (Barista, \$15,000). Each row has two icons in the 'Acciones' column: a pencil for editing and a trash can for deleting. A small circular help icon with a question mark is located in the bottom right corner.

Nombre	Puesto	Salario	Acciones
María García	Barista	\$15,000	 
Juan Pérez	Gerente	\$25,000	 
Ana Martínez	Cajero	\$12,000	 
Carlos López	Barista	\$15,000	 

Registrar Empleado

Nombre

KAREN ALEJANDRA HERRERA VILLALPANDO

Puesto

TESTER

Salario

20000

Guardar

Coffeet

Panel de Administración

+ Nuevo Empleado

Nombre	Puesto	Salario	Acciones
María García	Barista	\$15,000	 
Juan Pérez	Gerente	\$25,000	 
Ana Martínez	Cajero	\$12,000	 
Carlos López	Barista	\$15,000	 
KAREN ALEJANDRA HERRERA VILLALPANDO	TESTER	20000	 

1.3.3.2 Interfaces de hardware

El sistema es una aplicación de escritorio local acoplada a un backend contenerizado y no interactúa directamente con hardware específico o especializado (como lectores biométricos de asistencia o impresoras de tickets de caja). La interfaz se despliega directamente en el equipo del administrador. La conectividad local entre los componentes es gestionada por el

sistema operativo anfitrión y el motor de contenedores. El hardware de la computadora cliente debe contar con recursos suficientes de procesamiento y memoria RAM para ejecutar el motor de contenedores en segundo plano (soportando el servidor y la base de datos simultáneamente) junto con la aplicación de escritorio de manera fluida.

1.3.3.3 Interfaces de Software

- **Aplicación Cliente de Escritorio:** Interfaz gráfica nativa que requiere el entorno de ejecución correspondiente (por ejemplo, Java Runtime Environment) instalado en el sistema operativo del administrador para su correcta visualización y procesamiento local.
- **Servidor Backend (API REST):** Entorno lógico desarrollado en el lenguaje Go (Golang) que se ejecuta de forma aislada dentro de un contenedor. Es el encargado de recibir las peticiones de la interfaz y procesar las reglas de negocio de la gestión de personal.
- **Sistema de Gestión de Base de Datos (DBMS):** Servidor MySQL que opera de manera persistente dentro de su propio contenedor independiente para almacenar y organizar la tabla de empleados. La comunicación con el backend de Go se realiza a través de controladores (*drivers*) SQL nativos.
- **Entorno de Orquestación (Docker Compose):** Herramienta de software a nivel de sistema operativo encargada de definir, coordinar y levantar la infraestructura contenerizada del backend (servidor y base de datos) en la máquina local de forma automatizada.

1.3.3.4 Interfaces de Comunicaciones

Toda la comunicación entre la aplicación de escritorio y la API REST del backend se realiza dentro de la misma máquina o red interna (Localhost) a través del protocolo HTTP, empleando exclusivamente el formato JSON para el intercambio y transporte estructurado de datos. Al omitirse el inicio de sesión, el flujo de comunicación no transfiere credenciales de acceso ni requiere cifrado HTTPS para este fin. Por otro lado, la interconexión entre el servidor Go y el contenedor de MySQL se gestiona mediante el puente de red interno de Docker a través de los puertos estándar del DBMS (por ejemplo, el puerto 3306). La arquitectura del sistema depende de la disponibilidad y correcta asignación de los puertos locales del equipo anfitrión, funcionando de manera autónoma y sin requerir acceso a la red de Internet global.

1.3.3.5 Requerimientos funcionales

Esta sección incluye los requerimientos que especifican las acciones fundamentales del software que considera el sistema de Gestión de Empleados enfocado en el sector cafetero.

ID	RF1
Título	Registro de empleados (Alta)

Descripción	El sistema debe permitir registrar nuevos empleados capturando su nombre completo, puesto (ej. Barista, Cajero) y salario a través de un formulario en la aplicación de escritorio.
Intención	Crear el registro inicial del personal en la base de datos de la cafetería de forma centralizada.
Prioridad	Alta
Liberación	Versión 1.0
Dependencia	Contenedores de API REST y Base de Datos activos.

ID	RF2
Título	Consulta de plantilla laboral (Lectura)
Descripción	El sistema debe desplegar en la pantalla principal una tabla dinámica que muestre la lista completa y actualizada de todos los empleados registrados en el sistema.
Intención	Proveer al administrador una visualización general y en tiempo real del estado del personal.
Prioridad	Alta
Liberación	Versión 1.0
Dependencia	RF1

ID	RF3
Título	Actualización de datos del empleado (Edición)
Descripción	El sistema debe permitir seleccionar el registro de un empleado existente y modificar su información detallada (ajuste de salario o cambio de rol/puesto).
Intención	Mantener la información del personal precisa y actualizada ante promociones o cambios administrativos.
Prioridad	Alta
Liberación	Versión 1.0
Dependencia	RF1, RF2

ID	RF4
Título	Baja de personal (Eliminación)
Descripción	El sistema debe permitir eliminar de manera definitiva el registro de un empleado de la base de datos mediante un botón de acción en la interfaz de escritorio.
Intención	Depurar la base de datos para reflejar únicamente al personal que se encuentra activo en la cafetería.
Prioridad	Alta
Liberación	Versión 1.0
Dependencia	RF1, RF2

1.3.3.6 Requerimientos de rendimiento

ID	RNF1
Título	Rendimiento de peticiones HTTP
Descripción	La consulta general de la plantilla laboral (Read) debe devolver la lista completa desde la API en Go a la interfaz de escritorio en menos de 1 segundo.
Intención	Asegurar rapidez en la visualización de datos y evitar que la aplicación de escritorio se "congele" esperando respuesta.
Prioridad	Alta
Liberación	Versión 1.0
Dependencia	RF2 (Consulta de plantilla)

ID	RNF2
Título	Capacidad de procesamiento local
Descripción	El sistema y los contenedores Docker deben manejar las operaciones CRUD diarias sin degradación en el rendimiento del equipo anfitrión.
Intención	Garantizar la fluidez del sistema en la computadora de la cafetería sin saturar la memoria RAM.
Prioridad	Alta
Liberación	Versión 1.0
Dependencia	Global (Todos los RF)

1.3.3.7 Restricciones de diseño

ID	RNF3
Título	Arquitectura y Contenerización
Descripción	El backend (Servidor Go y Base de Datos MySQL) debe estar estrictamente contenerizado utilizando Docker y ejecutarse exclusivamente en un entorno Localhost.
Intención	Estandarizar el despliegue del servidor y aislar la base de datos del sistema operativo anfitrión.
Prioridad	Alta
Liberación	Versión 1.0
Dependencia	Arquitectura general del sistema

ID	RNF4
Título	Compatibilidad de Sistema Operativo
Descripción	La aplicación cliente debe ser un ejecutable nativo de escritorio compatible con el sistema operativo de la computadora de la cafetería (ej. Windows).
Intención	Asegurar que el administrador pueda ejecutar el software localmente sin requerir navegadores web.
Prioridad	Alta

Liberación	Versión 1.0
Dependencia	Interfaz de usuario

1.3.3.8 Atributos de software

ID	RNF5
Título	Usabilidad de Interfaz
Descripción	La interfaz de escritorio debe ser intuitiva. Un administrador sin experiencia técnica previa debe poder dar de alta a un nuevo empleado en un máximo de 2 minutos.
Intención	Asegurar facilidad de uso y agilizar los procesos administrativos de la cafetería.
Prioridad	Alta
Liberación	Versión 1.0
Dependencia	RF1, RF2

ID	RNF6
Título	Mantenibilidad del Código
Descripción	El código fuente de la API REST en Go y del cliente de escritorio debe estar documentado. La estructura debe seguir un patrón modular que separe la interfaz de la lógica de red.
Intención	Facilitar futuras correcciones de errores, revisión académica o adición de nuevas rutas en la API.
Prioridad	Alta
Liberación	Versión 1.0
Dependencia	Arquitectura general

ID	RNF7
Título	Disponibilidad Local
Descripción	El sistema debe estar disponible para su uso inmediato siempre que el equipo anfitrión esté encendido y el servicio de Docker esté corriendo en segundo plano.
Intención	Garantizar el acceso a la gestión de empleados durante toda la jornada laboral de la cafetería.
Prioridad	Alta
Liberación	Versión 1.0
Dependencia	Global (Todos los RF)

1.4 Priorización y plan de liberación

En el desarrollo la aplicación se contemplan diferentes versiones y actualizaciones, lo cual deriva en la necesidad de contar con una forma de dividir los requerimientos en diferentes etapas y con la priorización de esos requerimientos. Esta sección permite analizar algunos métodos de priorización y sugiere un plan para su liberación.

Matriz de trazabilidad y estabilidad

ID	Nombre del Requerimiento	Prioridad	Dependencias	Estabilidad
RF1	Registro de empleados (Alta)	Alta	Contenedores (Docker) activos	Alta
RF2	Consulta de plantilla laboral (Lectura)	Alta	RF1	Alta
RF3	Actualización de datos del empleado (Edición)	Alta	RF1, RF2	Alta
RF4	Baja de personal (Eliminación)	Alta	RF1, RF2	Alta
RNF1	Rendimiento de peticiones HTTP	Alta	RF2	Media
RNF2	Capacidad de procesamiento local	Media	Global (Todos los RF)	Alta
RNF3	Arquitectura y Contenerización	Alta	Arquitectura general	Alta
RNF4	Compatibilidad de Sistema Operativo	Alta	Interfaz de usuario	Alta
RNF5	Usabilidad de Interfaz	Alta	RF1, RF3	Media
RNF6	Mantenibilidad del Código	Alta	Arquitectura general	Alta
RNF7	Disponibilidad Local	Alta	Global (Todos los RF)	Alta

1.5 Descomposición del Sistema en Subsistemas

1.5.1 Subsistema de Presentación (Cliente de Escritorio)

Es la interfaz gráfica con la que interactúa el administrador. Se encarga de capturar las entradas del usuario (formularios de registro, clics en botones de editar/eliminar) y mostrar los datos en pantalla (tabla de empleados).

1.5.2 Subsistema de Lógica de Negocio (API REST en Go)

Actúa como el puente o intermediario. Recibe las peticiones HTTP del cliente de escritorio, procesa las reglas de negocio (validar que el salario sea un número, etc.) y decide qué hacer con la información.

1.5.3 Subsistema de Persistencia de Datos (Base de Datos)

Compuesto por el gestor MySQL. Su única responsabilidad es almacenar, consultar, actualizar y eliminar de forma segura los registros de los empleados en sus tablas correspondientes.

1.5.4 Subsistema de Orquestación (Infraestructura Docker)

Es el entorno que encapsula y ejecuta el backend (Go y MySQL) en contenedores aislados dentro del sistema operativo local (Localhost), garantizando que los servicios se comuniquen entre sí a través de puertos internos.

1.6 Objetivos

Objetivo General	
Desarrollar e implementar un sistema local cliente-servidor llamado "Coffeet" para la administración eficiente de la plantilla laboral de una cafetería, con el fin de digitalizar, centralizar y agilizar los registros del personal mediante operaciones en tiempo real desde una aplicación de escritorio.	
Objetivos Específicos	
Gestión de Personal (CRUD)	Proveer las funciones necesarias en la interfaz gráfica para crear (alta), consultar, actualizar y eliminar (baja) registros de empleados capturando sus atributos básicos: nombre, puesto (ej. Barista) y salario.
Comunicación e Integración	Implementar un servidor backend (API REST en Go) que reciba peticiones HTTP y gestione de forma eficiente el intercambio estructurado de datos en formato JSON entre la aplicación de escritorio y la base de datos.
Persistencia de Datos	Almacenar, organizar y mantener la integridad de la información de los empleados utilizando un sistema de gestión de bases de datos relacional (MySQL).
Despliegue y Orquestación	Configurar un entorno de ejecución local automatizado mediante el uso de contenedores (Docker) para garantizar el levantamiento simultáneo y el aislamiento técnico del servidor web y la base de datos.

--	--

1.7. Diccionario de Datos

Nombre del Campo	Restricciones	Tipo de Dato	Longitud	Obligatorio	Descripción
emp_no	PK / FK, NOT NULL	INT	11	Sí	Identificador único del empleado. Es llave primaria (PK) en empleados y foránea (FK) en puestos y salarios.
birth_date	NOT NULL	DATE	-	Sí	Fecha de nacimiento del trabajador (Formato YYYY-MM-DD).
first_name	NOT NULL	VARCHAR	50	Sí	Nombre(s) del empleado.
last_name	NOT NULL	VARCHAR	50	Sí	Apellido(s) del empleado.
gender	NOT NULL	ENUM	1	Sí	Género del empleado (ej. 'M' o 'F').
hire_date	NOT NULL	DATE	-	Sí	Fecha oficial en la que el empleado fue contratado.
id	PK, AUTO_INCREMENT	INT	11	Sí	Identificador único para el registro de las credenciales de acceso.
correo	UNIQUE, NOT NULL	VARCHAR	100	Sí	Correo electrónico de la cuenta, utilizado para el inicio de sesión.

contrasena	NOT NULL	VARCHAR	255	Sí	Contraseña de acceso a la aplicación.
rol	DEFAULT 'empleado'	VARCHAR	-	Sí	Nivel de privilegio y acceso en el sistema (ej. 'admin' o 'empleado').
empleado_id	FK, NULL	INT	11	Sí	Llave foránea que vincula la cuenta digital con el ID físico del empleado.
title	FK, NULL	VARCHAR	50	Sí	Nombre del cargo operativo dentro de la cafetería (ej. "Barista").
salary	FK, NULL	INT	11	Sí	Monto económico percibido por el trabajador.
from_date	FK, NULL	DATE	-	Sí	Fecha de inicio a partir de la cual entra en vigor un puesto o salario.
to_date	FK, NULL	DATE	-	Sí	Fecha en la que finaliza un puesto o salario. Utiliza '9999-01-01' si es el actual.

1.8 Tablas y normalización

1.8.1. Tabla: employees

Descripción: Almacena la información personal básica y el identificador físico de cada trabajador de la cafetería.

Fecha de creación: Mayo de 2026.

Campo	Tipo	Longitud	Descripción
emp_no	Numérico	11	Identificador único del empleado. GO
birth_date	Fecha	-	Fecha de nacimiento del empleado (YYYY-MM-DD). GO
first_name	Texto	50	Nombre(s) de la persona. GO
last_name	Texto	50	Apellido(s) de la persona. GO
gender	Texto	1	Género del empleado ('M' o 'F'). GO
hire_date	Fecha	-	Fecha en la que el empleado fue contratado. GO

Relaciones: Empleado con Puestos (emp_no FK en titles) y Salarios (emp_no FK en salaries).

Campo Clave: emp_no **Reglas de negocio:**

- El identificador emp_no no puede duplicarse y se genera secuencialmente (último ID + 1).
- Todos los campos personales son obligatorios al momento de registrar un nuevo elemento en la aplicación.

1.8.2. Tabla: usuarios

Descripción: Gestiona las credenciales de acceso digital y los niveles de privilegio (roles) para iniciar sesión en la aplicación móvil.

Fecha de creación: Mayo de 2026.

Campo	Tipo	Longitud	Descripción
id	Numérico	11	Identificador único de la credencial. DOCX + 1
correo	Texto	100	Correo electrónico de la cuenta. DOCX + 1
contrasena	Texto	255	Contraseña de acceso a la aplicación. DOCX + 1
rol	Texto	20	Nivel de acceso del usuario (ej. 'admin' o 'empleado'). DOCX + 1
empleado_id	Numérico	11	ID físico del empleado al que pertenece. DOCX + 1

Relaciones: Usuario con Empleado (empleado_id FK hacia employees.emp_no). **Campo**

Clave: id **Reglas de negocio:**

- El correo debe ser único y no puede repetirse en el sistema.
- El usuario con rol "admin" puede no estar ligado a un trabajador físico (su empleado_id puede ser NULL).
- El administrador no puede borrar su propio usuario mientras esté en sesión.

1.8.3. Tabla: titles

Descripción: Mantiene el registro del puesto o cargo operativo asignado a cada empleado dentro de la cafetería.

Fecha de creación: Mayo de 2026.

Campo	Tipo	Longitud	Descripción
emp_no	N Numérico	11	Identificador del empleado al que pertenece el puesto. GO
title	Texto	50	Nombre del cargo (ej. "Barista", "Supervisor"). GO
from_date	Fecha	-	Fecha en la que asumió este cargo. GO
to_date	Fecha	-	Fecha en la que finaliza el cargo. GO

Relaciones: Puesto con Empleado (emp_no FK hacia employees.emp_no). **Campo Clave:** emp_no **Reglas de negocio:**

- Cada puesto debe estar obligatoriamente vinculado a un empleado existente.
- Para indicar que el puesto es el **actual** y está activo, el campo to_date debe tener el valor '9999-01-01'.

1.8.4. Tabla: salaries

Descripción: Registra el historial de sueldos o salarios económicos asignados a los trabajadores.

Fecha de creación: Mayo de 2026.

Campo	Tipo	Longitud	Descripción
emp_no	N Numérico	11	Identificador del empleado al que pertenece el salario. GO
salary	N Numérico	11	Monto económico percibido por el trabajador. GO
from_date	Fecha	-	Fecha a partir de la cual entra en vigor el salario. GO
to_date	Fecha	-	Fecha en la que finaliza el salario. GO

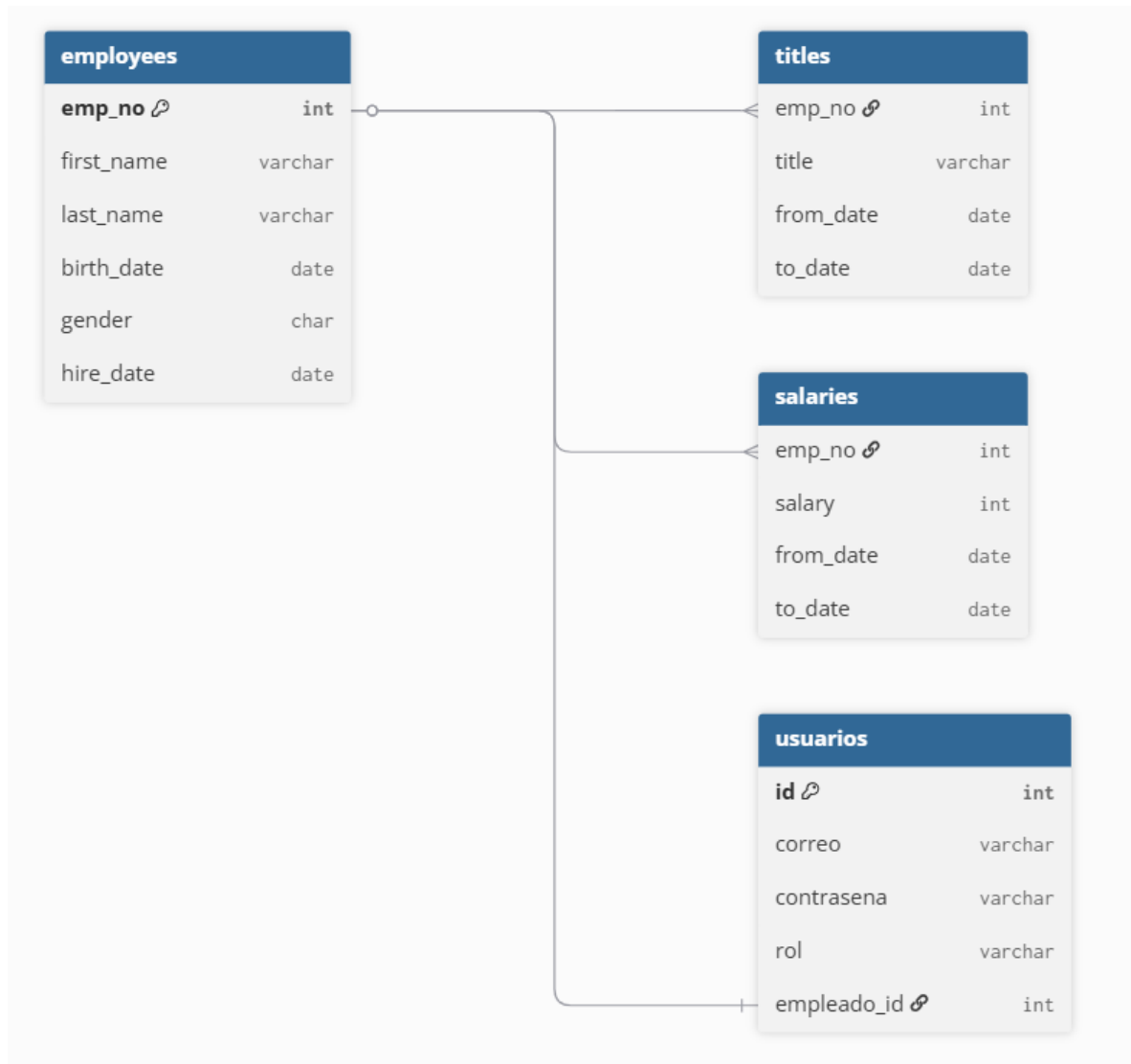
Relaciones: Salario con Empleado (emp_no FK hacia employees.emp_no). **Campo Clave:** emp_no **Reglas de negocio:**

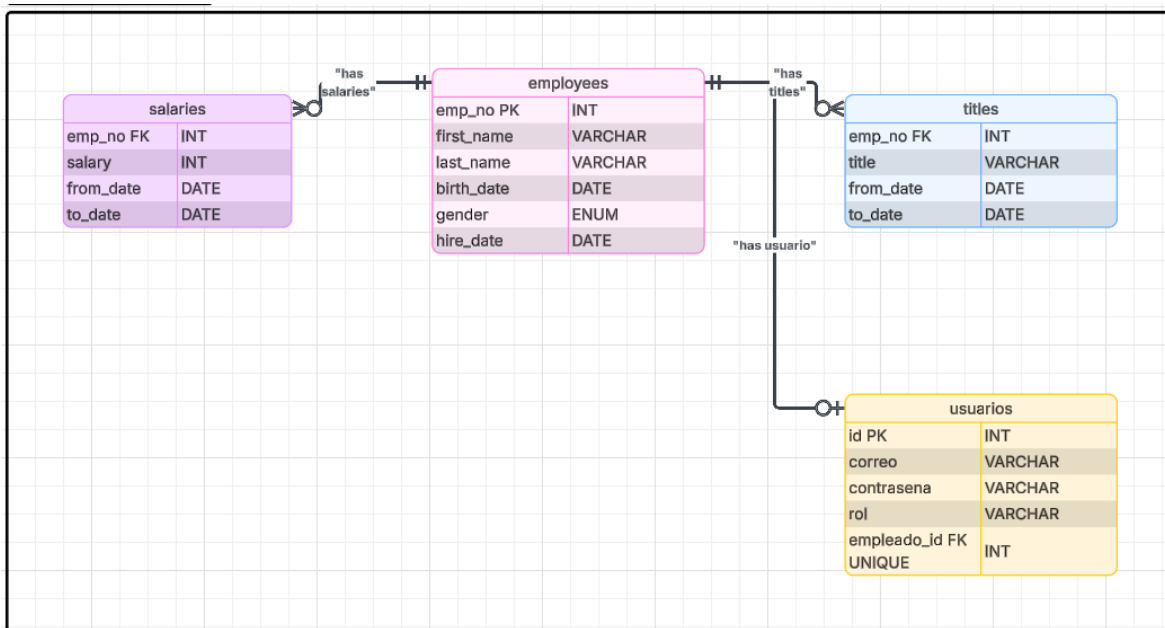
- Solo los usuarios con rol "admin" pueden modificar esta tabla desde la aplicación.

- Para indicar que el salario es el **actual**, el campo `to_date` debe tener el valor '9999-01-01'.

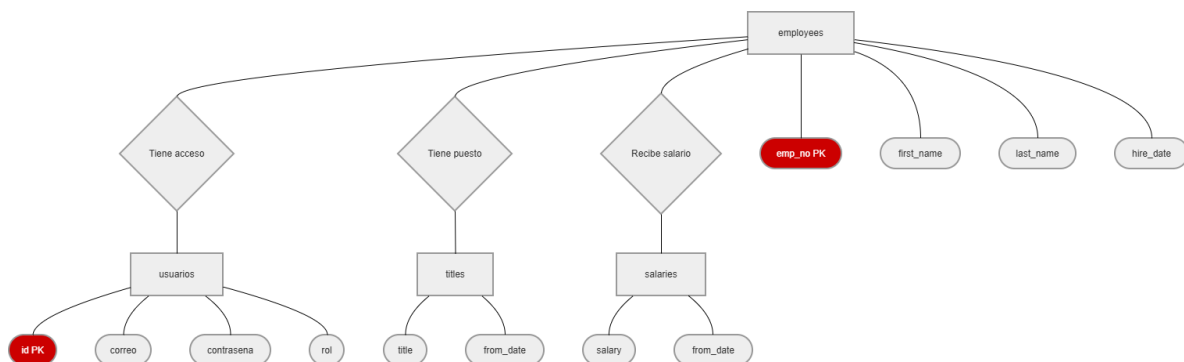
1.9 DIAGRAMAS

1.9.1 Diagrama de clase





1.9.2 Diagrama Entidad Relación



1.10 Normalización

Para garantizar la integridad de los datos, evitar la redundancia y prevenir anomalías de inserción o actualización en el sistema "Coffeet", se aplicaron las reglas de normalización a la base de datos hasta alcanzar la Tercera Forma Normal (3FN).

Partimos de un supuesto diseño inicial no normalizado (Tabla Universal) donde se registraba toda la información junta, lo cual generaba duplicidad de datos personales cada vez que un empleado cambiaba de sueldo o puesto:

- **ReporteGeneral** (emp_no, first_name, last_name, birth_date, gender, hire_date, title, title_from_date, salary, salary_from_date, correo, contrasena, rol)

1.10.1 Primera Forma Normal (1FN)

Regla: Eliminar grupos repetidos y asegurar que todos los atributos contengan valores atómicos. Debe existir una clave primaria única. **Aplicación:** Se separó el historial de datos que generan multiplicidad (los sueldos y los puestos operativos) de la entidad biológica del trabajador. Se definieron las llaves primarias compuestas para los historiales.

- **employees** (**emp_no**, first_name, last_name, birth_date, gender, hire_date, correo, contrasena, rol)
- **titles** (**emp_no**, **from_date**, title, to_date)
- **salaries** (**emp_no**, **from_date**, salary, to_date)

1.10.2. Segunda Forma Normal (2FN)

Regla: Cumplir con la 1FN y eliminar las dependencias parciales. Ningún atributo no clave debe depender de solo una parte de la clave primaria compuesta. **Aplicación:** En nuestro modelo, las tablas **titles** y **salaries** tienen llaves compuestas (**emp_no** + **from_date**). El monto del salario y el nombre del puesto dependen de la combinación completa de "qué empleado es" y "a partir de qué fecha", por lo que ya cumplen con la 2FN de manera natural. La tabla **employees** tiene una llave primaria simple, por lo que también cumple la 2FN.

- **employees** (**emp_no**, first_name, last_name, birth_date, gender, hire_date, correo, contrasena, rol)
- **titles** (**emp_no**, **from_date**, title, to_date)
- **salaries** (**emp_no**, **from_date**, salary, to_date)

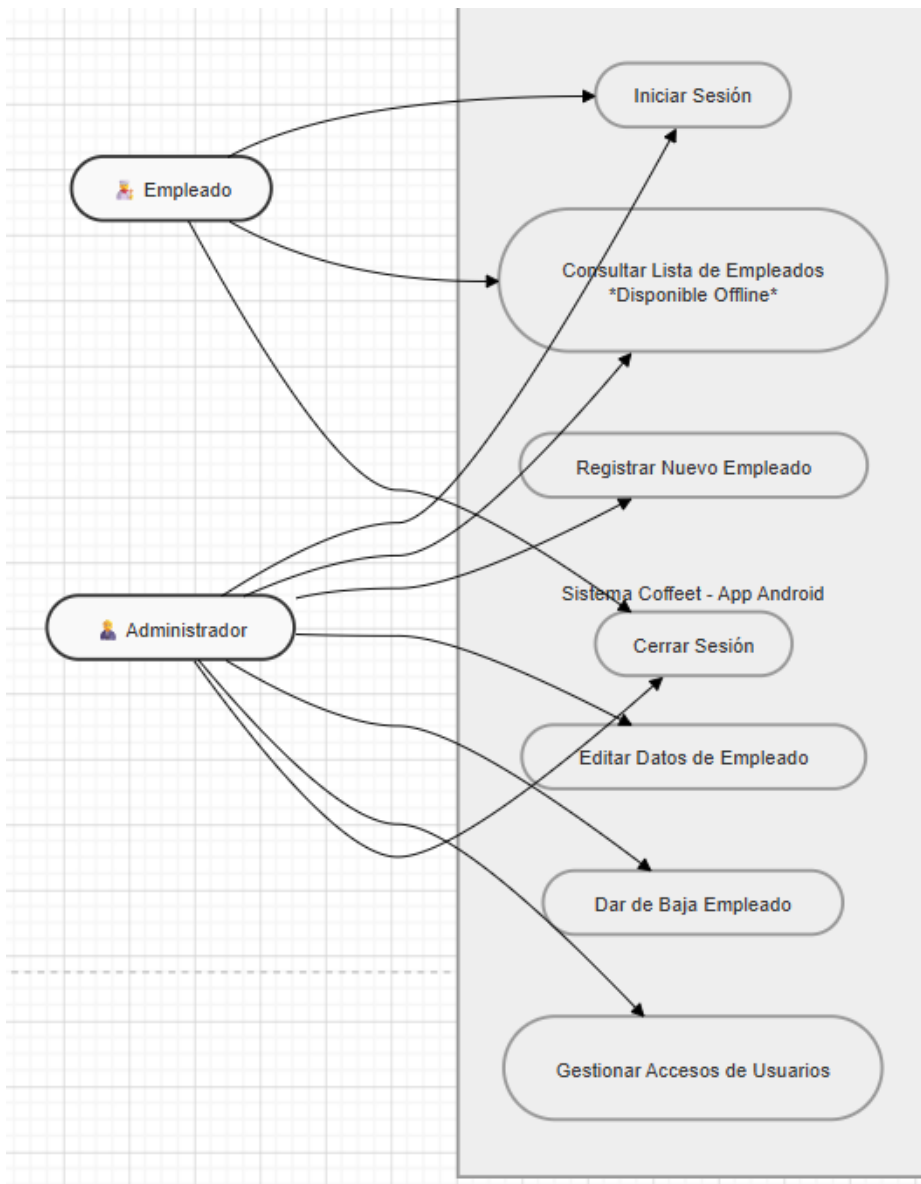
1.10.3. Tercera Forma Normal (3FN)

Regla: Cumplir con la 2FN y eliminar las dependencias transitivas. Ningún atributo no clave debe depender de otro atributo que tampoco sea clave. **Aplicación:** En la tabla **employees**, los campos **correo**, **contrasena** y **rol** representan las credenciales de acceso a la aplicación, no atributos intrínsecos de la persona. La contraseña y el rol dependen lógicamente del correo (cuenta digital), no del número de empleado físico (**emp_no**). Además, el requerimiento indica que el Administrador puede no estar ligado a un empleado físico. Por lo tanto, se extraen estos campos a una tabla independiente llamada **usuarios**, vinculándola a través de una llave foránea que permite valores nulos.

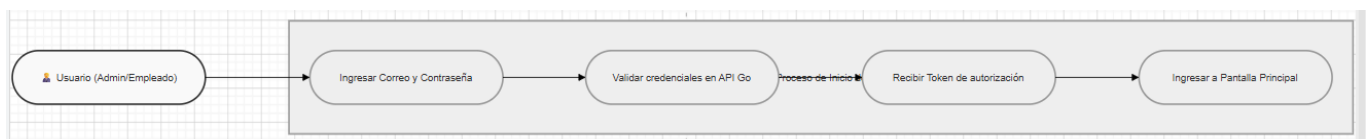
Estructura Final (3FN):

- **employees** (**emp_no**, first_name, last_name, birth_date, gender, hire_date)
- **usuarios** (**id**, correo, contrasena, rol, *empleado_id*)
- **titles** (**emp_no**, **from_date**, title, to_date)
- **salaries** (**emp_no**, **from_date**, salary, to_date)

1.11 Casos de uso y diagramas de casos de uso

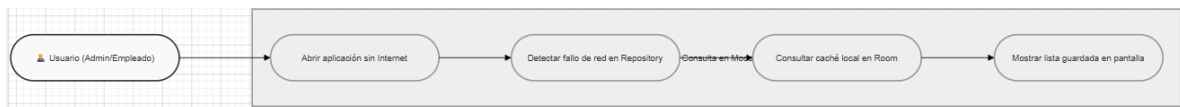


ID: CU-01	Nombre: Registrar Nuevo Empleado
Actor Principal:	Administrador
Descripción:	Permite al administrador dar de alta a un nuevo trabajador en el sistema de la cafetería, guardando sus datos en la base de datos central.
Precondiciones:	El actor debe tener la aplicación Android abierta, contar con conexión a internet y haber iniciado sesión con un token de autorización válido con el rol "admin".
Flujo Principal:	1. El Administrador selecciona la opción "Agregar Empleado (+)" en la pantalla principal. 2. El sistema muestra el formulario <code>EmpleadoFormViewModel</code>. 3. El Administrador ingresa nombre, apellido, puesto y salario. 4. El Administrador presiona "Guardar". 5. El sistema empaqueta los datos en un JSON y hace una petición POST a la API. 6. La API inserta el registro en las tablas <code>employees</code>, <code>titles</code> y <code>salaries</code> de MySQL. 7. El sistema muestra el mensaje de éxito y actualiza la lista visual.
Flujos Alternativos:	A. Falla de conexión: Si el servidor está caído o no hay internet, la app atrapa el error y muestra un <i>Snackbar</i> indicando "Error de conexión con el servidor". El registro se cancela. B. Datos incompletos: Si falta algún campo, el botón de guardar se desactiva o marca el campo en rojo.
Postcondiciones:	El nuevo empleado queda registrado en la base de datos y la lista de la pantalla principal (<code>EmpleadoScreen</code>) se recarga incluyendo el nuevo integrante.



ID: CU-02	Nombre: Iniciar Sesión de Usuario
Actor Principal:	Administrador / Empleado

Descripción:	Permite al usuario autenticarse en el sistema mediante sus credenciales (correo y contraseña) para acceder a las funciones de la aplicación según su nivel de privilegios.
Precondiciones:	El dispositivo debe contar con conexión a internet para validar las credenciales contra el servidor de la nube (Aiven). El usuario debe existir previamente en la tabla usuarios.
Flujo Principal:	1. El actor abre la aplicación y visualiza la pantalla de Login. 2. El actor ingresa su correo electrónico y contraseña. 3. El actor presiona el botón "Iniciar Sesión". 4. El sistema empaqueta los datos (LoginRequestDto) y envía una petición a la API en Go. 5. La API verifica que el correo exista y que la contraseña coincida en la base de datos. 6. La API devuelve una respuesta exitosa con el rol del usuario ("admin" o "empleado") y un Token. 7. El sistema guarda el Token localmente (en DataStore) y redirige al usuario a la Pantalla Principal.
Flujos Alternativos:	A. Credenciales incorrectas: Si el correo o la contraseña no coinciden, la API devuelve un error y la aplicación muestra un mensaje indicando "Correo o contraseña incorrectos". B. Error de red: Si no hay internet, el sistema atrapa el error (try-catch) y alerta al usuario sobre el fallo de conexión con el servidor.
Postcondiciones:	El usuario ingresa al sistema y la aplicación habilita o deshabilita funciones visuales (como los botones de agregar/eliminar) dependiendo del rol recibido.



ID: CU-03	Nombre: Consultar Lista de Empleados (Offline)
------------------	---

Actor Principal:	Administrador / Empleado
Descripción:	Permite a cualquier usuario visualizar el directorio del personal operativo guardado en la caché del teléfono, asegurando que la información operativa esté disponible incluso si se pierde la conexión a internet en la cafetería.
Precondiciones:	El usuario debe tener una sesión iniciada activa. El dispositivo debe haber sincronizado los datos con la API al menos una vez en el pasado para tener información guardada localmente.
Flujo Principal:	1. El actor navega a la pantalla principal (EmpleadoScreen). 2. El repositorio de datos (EmpleadoRepository) detecta una falla de red o la ausencia de conexión a internet. 3. El sistema consulta directamente la base de datos local del teléfono (Room). 4. El sistema extrae la lista de empleados guardada en la última sincronización. 5. La pantalla dibuja la lista reactiva mostrando nombres, puestos y salarios disponibles.
Flujos Alternativos:	A. Base de datos local vacía: Si es la primera vez que se usa la aplicación y no hay internet para la primera descarga, la pantalla mostrará un mensaje indicando "No hay datos disponibles, por favor conéctate a internet para sincronizar".
Postcondiciones:	El actor puede visualizar la información del personal, pero no puede realizar acciones de escritura (crear, editar o eliminar) hasta que se recupere la conexión con el servidor central.

1.12 Interfaz gráfica de usuario

La interfaz gráfica del sistema Coffeet fue desarrollada para dispositivos Android utilizando Jetpack Compose, proporcionando una experiencia intuitiva y fácil de utilizar para los administradores y empleados de la cafetería.

Pantalla de Inicio de Sesión

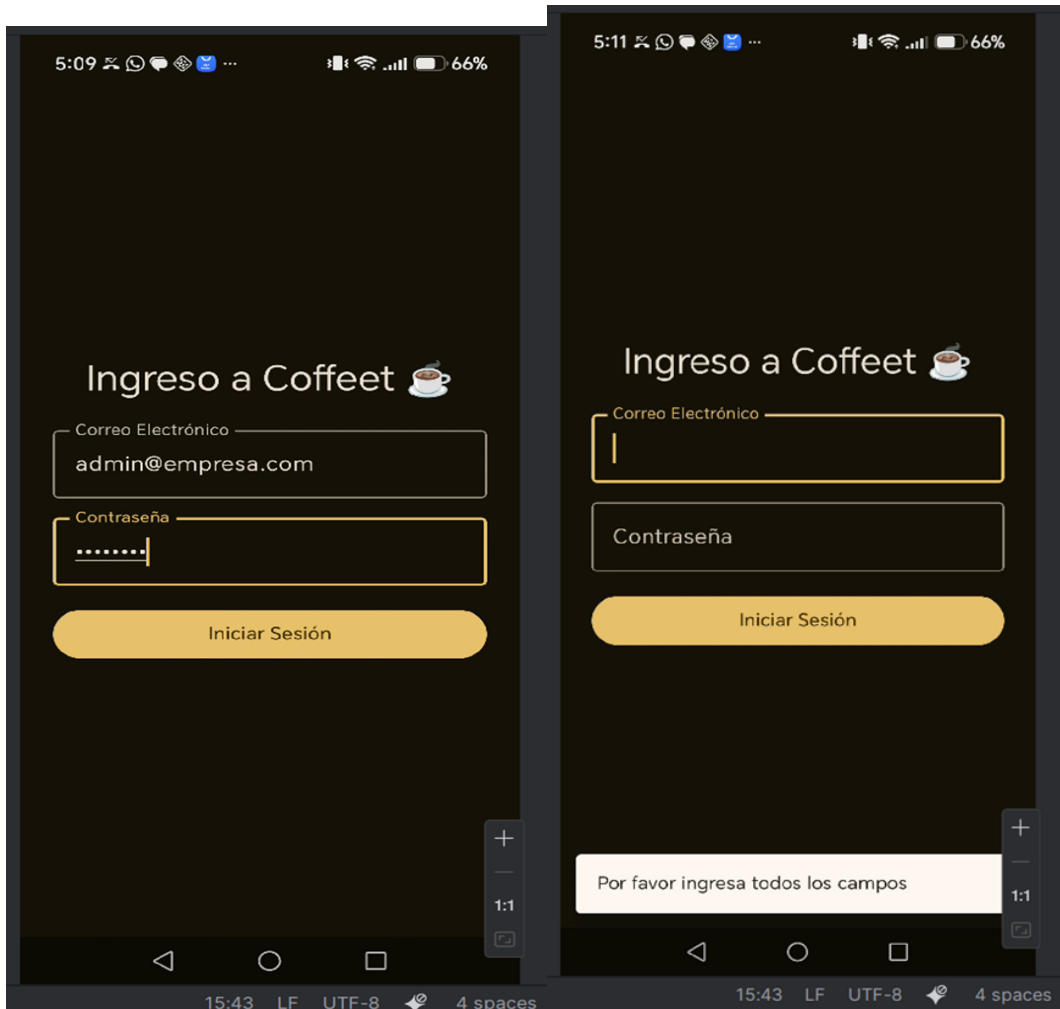
Esta pantalla permite la autenticación de los usuarios mediante correo electrónico y contraseña. El sistema valida las credenciales a través de la API REST y determina el nivel de acceso correspondiente según el rol asignado.

Funciones principales:

- Captura de correo electrónico.
- Captura de contraseña.

- Validación de credenciales.
- Acceso según rol de usuario.

Figura 1. Pantalla de inicio de sesión.



Pantalla Principal de Empleados

Después de iniciar sesión correctamente, el usuario accede a la pantalla principal donde se muestra la lista de empleados registrados.

Funciones principales:

- Visualización de empleados.

- Consulta de información laboral.
- Actualización automática de datos.
- Acceso a operaciones según permisos.

Figura 2. Pantalla principal de empleados.



Pantalla de Registro de Empleados

Disponible únicamente para usuarios con rol de administrador.

Permite registrar nuevos empleados mediante un formulario donde se capturan los siguientes datos:

- Nombre.
- Apellido.
- Puesto.
- Salario.
- Información laboral correspondiente.

Al finalizar el registro, la información es enviada a la API REST para su almacenamiento en la base de datos.

Figura 3. Pantalla de registro de empleados.

Datos del Personal

Nombre: Marco

Apellido: Frausto

Puesto: Repartidor

Salario: 30500

Credenciales de Acceso (API)

Correo Electrónico:

Contraseña de Acceso:

¿Es Administrador? ☐

Guardar Empleado y Credenciales

Por favor, completa todos los campos

Empleados Coffeet

ID	Nombre	Puesto	Salario	Acciones
500024	Jessica Serna	Ventas	\$50505	[Editar] [Eliminar]
500023	Angélica Hernández	Administradora	\$80504	[Editar] [Eliminar]
500022	Felipe Castillo	Pastor	\$100905	[Editar] [Eliminar]
500020	Gotardo Mata	Engineer		[Editar] [Eliminar]
500017	Kenia Os	Marketing		[Editar] [Eliminar]

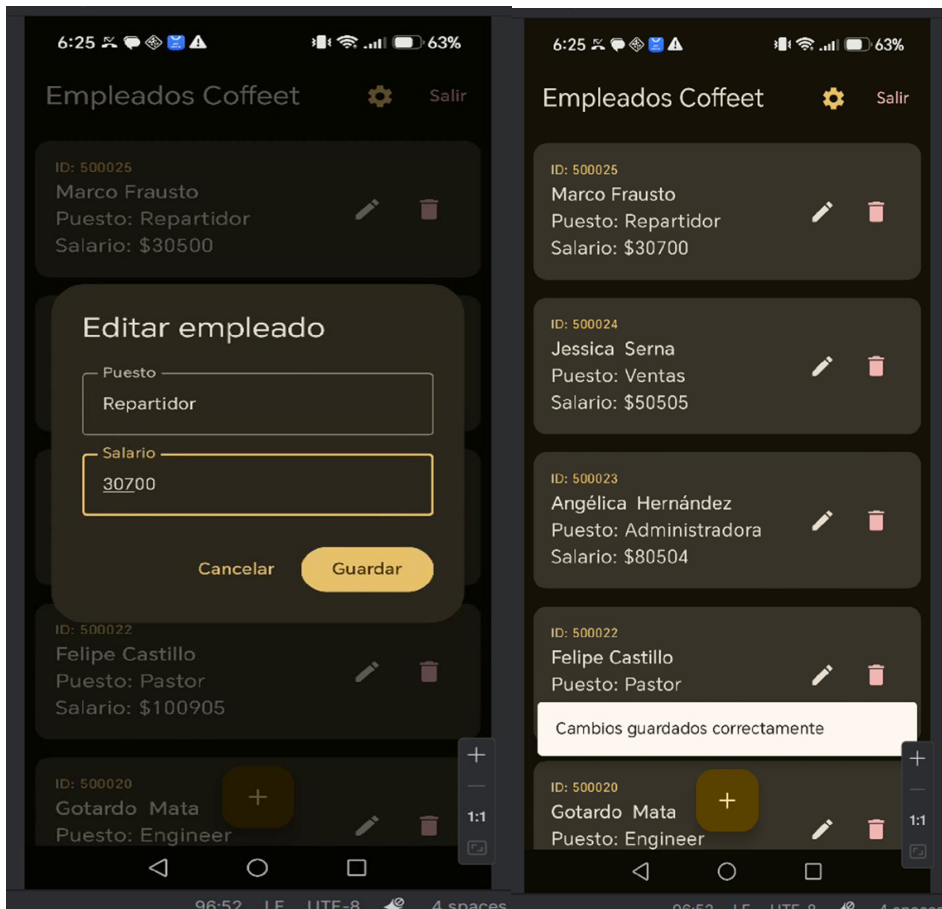
Empleado registrado correctamente

Pantalla de Edición de Empleados

Permite modificar la información de empleados previamente registrados.

Las modificaciones realizadas son enviadas al servidor para mantener actualizada la información almacenada.

Figura 4. Pantalla de edición de empleados.



Pantalla de Gestión de Usuarios

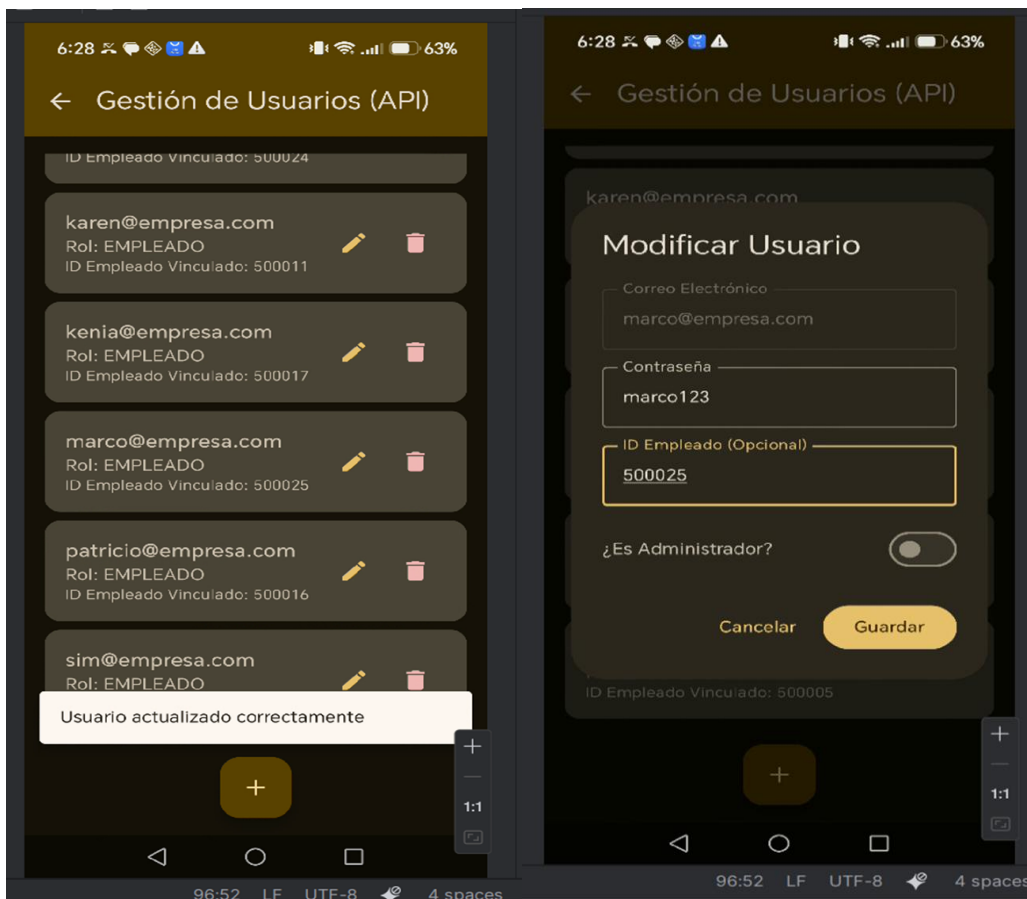
Permite administrar las cuentas de acceso al sistema.

Funciones:

- Crear usuarios.
- Modificar usuarios.
- Eliminar usuarios.
- Asignar roles.
- Vincular usuarios con empleados.

Esta funcionalidad se encuentra restringida exclusivamente para administradores.

Figura 5. Pantalla de gestión de usuarios.

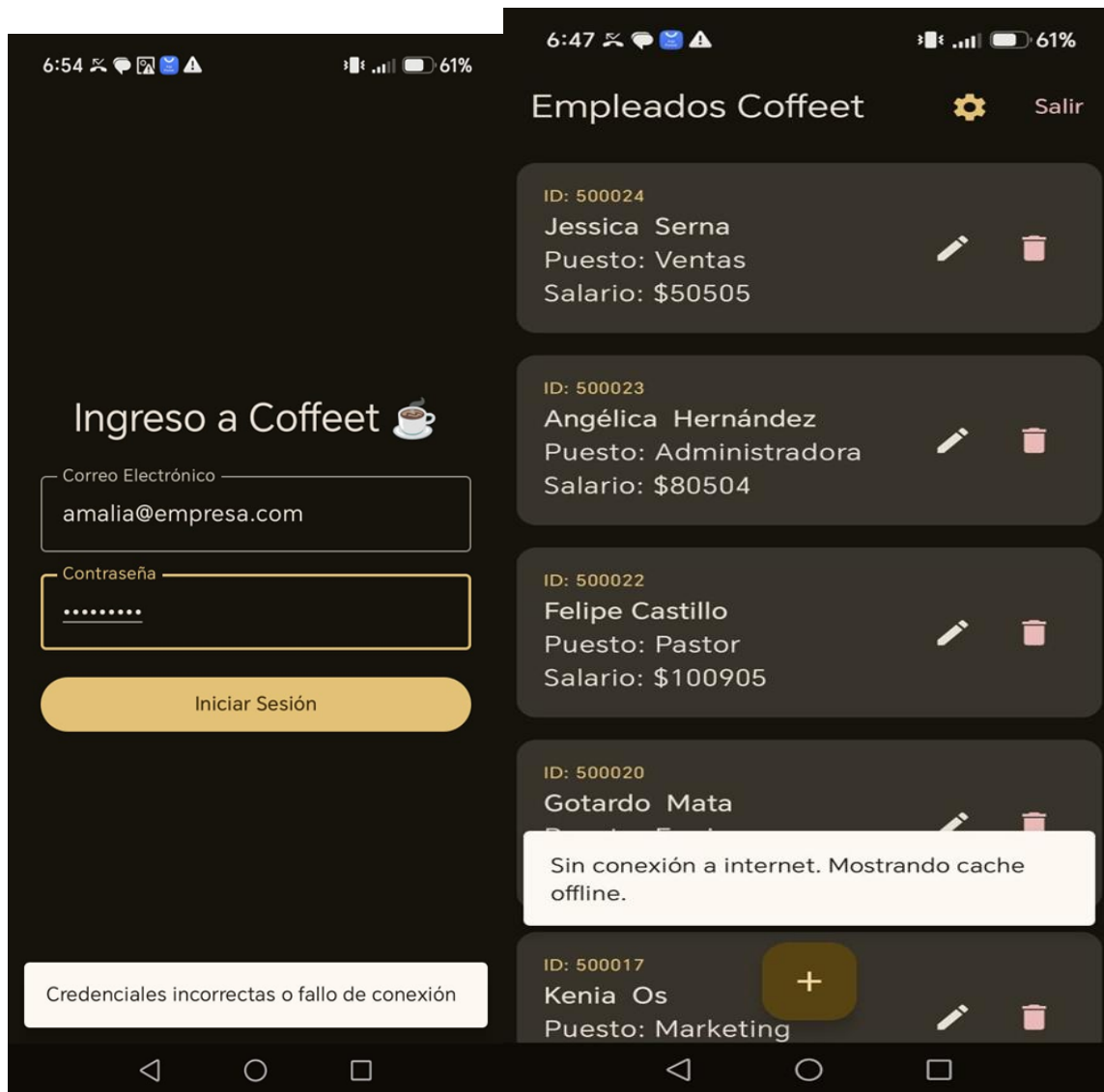


Interfaz en Modo Offline

Cuando no existe conexión a internet, la aplicación utiliza Room Database para mostrar información previamente sincronizada.

El usuario puede consultar la lista de empleados almacenada localmente, aunque las operaciones de escritura permanecen deshabilitadas hasta recuperar la conexión.

Figura 6. Consulta de empleados sin conexión.



1.13. Programación e implementación

El sistema Coffeet fue desarrollado siguiendo una arquitectura cliente-servidor que permite la comunicación entre una aplicación móvil Android y una API REST encargada del procesamiento de información.

Tecnologías Utilizadas

Aplicación Android

- Kotlin
- Jetpack Compose
- MVVM
- Clean Architecture

- Navigation Compose
- Hilt
- Retrofit
- Room Database
- DataStore

Backend

- Go (Golang)
- API REST
- JWT para autenticación

Base de Datos

- MySQL
- Servicio de alojamiento Aiven Cloud

Arquitectura Implementada

El proyecto fue desarrollado utilizando Clean Architecture y el patrón MVVM.

La estructura se encuentra dividida en tres capas principales:

Capa de Presentación

Responsable de la interacción con el usuario mediante pantallas desarrolladas con Jetpack Compose.

Incluye:

- Pantallas Compose.
- ViewModels.
- Navigation.
- StateFlow.

Capa de Dominio

Contiene la lógica de negocio del sistema.

Funciones principales:

- Validaciones.
- Reglas de negocio.
- Gestión de permisos.
- Control de acceso por roles.

Capa de Datos

Encargada de la comunicación con los servicios externos y almacenamiento local.

Componentes:

- Retrofit.
- Room Database.

- DataStore.
- Repositorios.
- DTOs.

Implementación del Backend

La API REST desarrollada en Go procesa las solicitudes provenientes de la aplicación móvil.

Entre sus responsabilidades se encuentran:

- Autenticación de usuarios.
- Gestión de empleados.
- Gestión de usuarios.
- Validación de credenciales.
- Generación de Tokens JWT.
- Comunicación con la base de datos MySQL.

Implementación del Sistema de Seguridad

La autenticación se realiza mediante Tokens JWT.

Después de un inicio de sesión exitoso:

1. El servidor genera un token.
2. El token es enviado a la aplicación.
3. DataStore almacena la sesión localmente.
4. OkHttp agrega automáticamente el token a las peticiones protegidas.

Además, el sistema implementa control de acceso basado en roles:

- Administrador.
- Empleado.

Implementación de Persistencia Local

Para garantizar disponibilidad parcial sin conexión a internet se implementó Room Database.

Esta base de datos local almacena:

- Información de empleados.
- Datos sincronizados previamente.

De esta forma el usuario puede consultar información aun cuando el servidor no se encuentre disponible.

1.14. Pruebas e implantación

Con el objetivo de verificar el correcto funcionamiento del sistema se realizaron diversas pruebas funcionales, de conectividad y de acceso.

Pruebas de Inicio de Sesión

Caso de prueba	Resultado
Credenciales validadas	correcto
Credenciales invalidas	correcto
Usuario inexistente	correcto
Error de conexión	correcto

El sistema valida adecuadamente las credenciales y muestra mensajes informativos cuando ocurre algún error.

Pruebas de Gestión de Empleados

Caso de prueba	Resultado
Registrar empleado	correcto
Consultar empleados	correcto
Actualizar empleado	correcto
Eliminar Empleado	correcto

Todas las operaciones CRUD fueron ejecutadas satisfactoriamente mediante la API REST.

Pruebas de Gestión de Usuarios

Caso de prueba	Resultado
Crear usuario	correcto
Modificar usuario	correcto
Eliminar usuario	correcto
Asignar usuario	correcto

El sistema restringe correctamente el acceso a estas funciones según el rol asignado.

Pruebas de Control de Roles

Caso de prueba	Resultado
Administrador puede gestionar empleados	correcto
Administrador puede gestionar usuarios	correcto
Empleado puede consultar información	correcto
Empleado no puede administrar usuarios	correcto

Los permisos fueron aplicados correctamente de acuerdo con las reglas de negocio establecidas.

Pruebas offline

Caso de prueba	Resultado
Consulta de empleados sin internet	correcto
Registro de empleados sin internet	correcto
Edición de empleados sin internet	correcto
Eliminación de empleados sin internet	correcto

Las pruebas demostraron que la aplicación mantiene acceso a información previamente sincronizada mediante Room Database.

Implantación del Sistema

La implantación consistió en la instalación de la aplicación Android en dispositivos compatibles y la conexión con la infraestructura compuesta por:

- API REST desarrollada en Go.
- Base de datos MySQL alojada en Aiven Cloud.
- Servicios de autenticación mediante JWT.

Posteriormente se realizaron pruebas integrales de funcionamiento verificando:

- Comunicación entre aplicación y servidor.
- Persistencia de datos.
- Manejo de sesiones.
- Control de roles.
- Funcionamiento offline.

Resultados Obtenidos

Los resultados obtenidos demostraron que el sistema cumple satisfactoriamente con los requerimientos funcionales y no funcionales establecidos para la gestión de empleados y usuarios de la cafetería, garantizando seguridad, disponibilidad de información y facilidad de uso.

4.3 Manual de Sistema

4.3.1 Introducción

El presente Manual de Sistema tiene como finalidad proporcionar las instrucciones necesarias para la instalación, configuración, operación y uso correcto del sistema Coffeet.

Este documento está dirigido a administradores y usuarios autorizados que utilizarán la aplicación para la gestión de empleados y usuarios dentro de la cafetería. Asimismo, sirve como guía de consulta para el mantenimiento y operación diaria del sistema.

4.3.2 Requisitos del Sistema

Requisitos de Hardware

Para garantizar un funcionamiento adecuado de la aplicación se recomienda contar con:

- Procesador Intel Core i5 o superior.
- Memoria RAM mínima de 8 GB.
- Almacenamiento disponible de al menos 500 MB.
- Conexión estable a Internet para la sincronización con el servidor.

Requisitos de Software

- Sistema operativo Android 8.0 o superior.
- Acceso a Internet.
- Servicios de Google Play habilitados.
- API REST desplegada y disponible.
- Base de datos MySQL activa.

4.3.3 Instalación de la Aplicación

1. Obtener el archivo APK de la aplicación Coffeet.
2. Transferir el archivo al dispositivo Android.
3. Habilitar la instalación desde fuentes desconocidas en caso de ser necesario.
4. Ejecutar el archivo APK.
5. Seguir el asistente de instalación.
6. Abrir la aplicación desde el menú de aplicaciones.

Una vez completada la instalación, el sistema mostrará la pantalla de inicio de sesión.

4.3.4 Inicio de Sesión

Para acceder al sistema:

1. Abrir la aplicación.
2. Introducir el correo electrónico registrado.
3. Introducir la contraseña correspondiente.
4. Presionar el botón **Iniciar Sesión**.
5. Esperar la validación de credenciales.

Si las credenciales son correctas, el sistema redireccionará automáticamente a la pantalla principal.

Si los datos son incorrectos, se mostrará un mensaje indicando que el correo o la contraseña no son válidos.

4.3.5 Pantalla Principal

La pantalla principal muestra el directorio de empleados registrados dentro del sistema.

Desde esta pantalla es posible:

- Consultar empleados.
- Actualizar la información mostrada.
- Acceder a las opciones disponibles según el rol del usuario.
- Visualizar información general del personal.

Los permisos visibles dependen del rol asignado durante el inicio de sesión.

4.3.6 Gestión de Empleados

Registrar Empleado

Esta función permite agregar un nuevo empleado al sistema.

Procedimiento:

1. Seleccionar la opción **Agregar Empleado**.
2. Capturar los datos solicitados.
3. Verificar la información ingresada.
4. Presionar el botón **Guardar**.
5. Esperar la confirmación del sistema.

Una vez completado el proceso, el empleado aparecerá en la lista general.

Consultar Empleados

Permite visualizar la información almacenada de todos los empleados registrados.

Procedimiento:

1. Ingresar a la pantalla principal.
2. Esperar la carga de información.
3. Revisar los registros mostrados.

La información se obtiene desde el servidor o desde la base de datos local cuando no existe conexión.

Editar Empleado

Permite modificar la información de un empleado existente.

Procedimiento:

1. Seleccionar el empleado deseado.
2. Presionar la opción **Editar**.
3. Modificar los campos necesarios.
4. Guardar los cambios realizados.

El sistema actualizará automáticamente la información en la base de datos.

Eliminar Empleado

Permite eliminar un registro de empleado.

Procedimiento:

1. Seleccionar el empleado.
2. Presionar la opción **Eliminar**.
3. Confirmar la operación.
4. Esperar la respuesta del sistema.

Una vez eliminado, el registro dejará de aparecer en la lista general.

4.3.7 Gestión de Usuarios

La administración de usuarios está disponible únicamente para cuentas con privilegios de administrador.

Las funciones disponibles son:

- Crear usuarios.
- Modificar usuarios.
- Eliminar usuarios.
- Asignar roles.
- Asociar usuarios con empleados.

Los usuarios con rol de empleado no tienen acceso a estas funciones.

4.3.8 Roles del Sistema

Administrador

Cuenta con acceso total al sistema.

Permisos:

- Consultar empleados.
- Registrar empleados.
- Editar empleados.
- Eliminar empleados.
- Gestionar usuarios.
- Asignar roles.
- Acceder a configuraciones administrativas.

Empleado

Cuenta con acceso limitado.

Permisos:

- Iniciar sesión.
- Consultar información autorizada.

Restricciones:

- No puede registrar empleados.
- No puede modificar empleados.
- No puede eliminar empleados.
- No puede administrar usuarios.

4.3.9 Funcionamiento Sin Conexión

La aplicación incorpora almacenamiento local para mantener la disponibilidad de información previamente sincronizada.

Funciones disponibles sin Internet

- Consulta de empleados almacenados localmente.
- Visualización de datos sincronizados anteriormente.

Funciones no disponibles sin Internet

- Inicio de sesión.
- Registro de empleados.
- Edición de empleados.
- Eliminación de empleados.
- Gestión de usuarios.
- Sincronización de información.

Cuando no exista conexión y no se disponga de datos locales, el sistema mostrará un mensaje informando que es necesario conectarse a Internet para realizar la primera sincronización.

4.3.10 Manejo de Errores

Error de conexión

Se presenta cuando la aplicación no puede comunicarse con el servidor.

Acción recomendada:

- Verificar la conexión a Internet.
- Intentar nuevamente más tarde.

Credenciales incorrectas

Se presenta cuando el correo o contraseña no coinciden con los registros existentes.

Acción recomendada:

- Verificar los datos capturados.
- Intentar nuevamente.

Error interno del servidor

Se presenta cuando ocurre una falla en la API o en la base de datos.

Acción recomendada:

- Esperar unos minutos.
- Contactar al administrador del sistema.

4.3.11 Seguridad

El sistema implementa mecanismos de seguridad para proteger la información de los usuarios.

Entre ellos se encuentran:

- Autenticación mediante credenciales.
- Gestión de permisos basada en roles.
- Uso de tokens de acceso.
- Almacenamiento local seguro de la sesión.
- Restricción de funciones según privilegios.

Además, la información administrativa se encuentra protegida y únicamente es accesible para usuarios autorizados.

4.3.12 Cierre de Sesión

Para finalizar una sesión activa:

1. Acceder al menú principal.
2. Seleccionar la opción **Cerrar Sesión**.
3. Confirmar la operación.

El sistema eliminará la sesión almacenada localmente y redireccionará nuevamente a la pantalla de inicio de sesión.

4.3.13 Recomendaciones de Uso

- Mantener una conexión estable a Internet para garantizar la sincronización de información.
- No compartir credenciales de acceso.
- Cerrar sesión al finalizar la jornada laboral.
- Mantener la aplicación actualizada.
- Verificar periódicamente que la información mostrada corresponda con los registros actuales.

4.3.14 Conclusión

El sistema Coffeet proporciona una herramienta centralizada para la administración de empleados y usuarios dentro de la cafetería. Su implementación permite optimizar la gestión de información, reducir errores operativos y facilitar el acceso seguro a los datos mediante una interfaz intuitiva y adaptada a las necesidades del negocio.

Conclusión

La realización de este proyecto nos permitió aplicar y reforzar conocimientos de desarrollo móvil, bases de datos y APIs REST en un caso práctico. A lo largo del proceso enfrentamos distintos retos que ayudaron a mejorar nuestras habilidades técnicas y de resolución de problemas.

Consideramos que el sistema Coffeet cumple con los objetivos planteados, ya que facilita la gestión de empleados y usuarios de una manera más organizada, segura y eficiente. Además, este proyecto representó una experiencia valiosa que contribuyó a nuestra formación profesional y al desarrollo de nuevas competencias en el área de software.